Data Structures and Algorithms I.

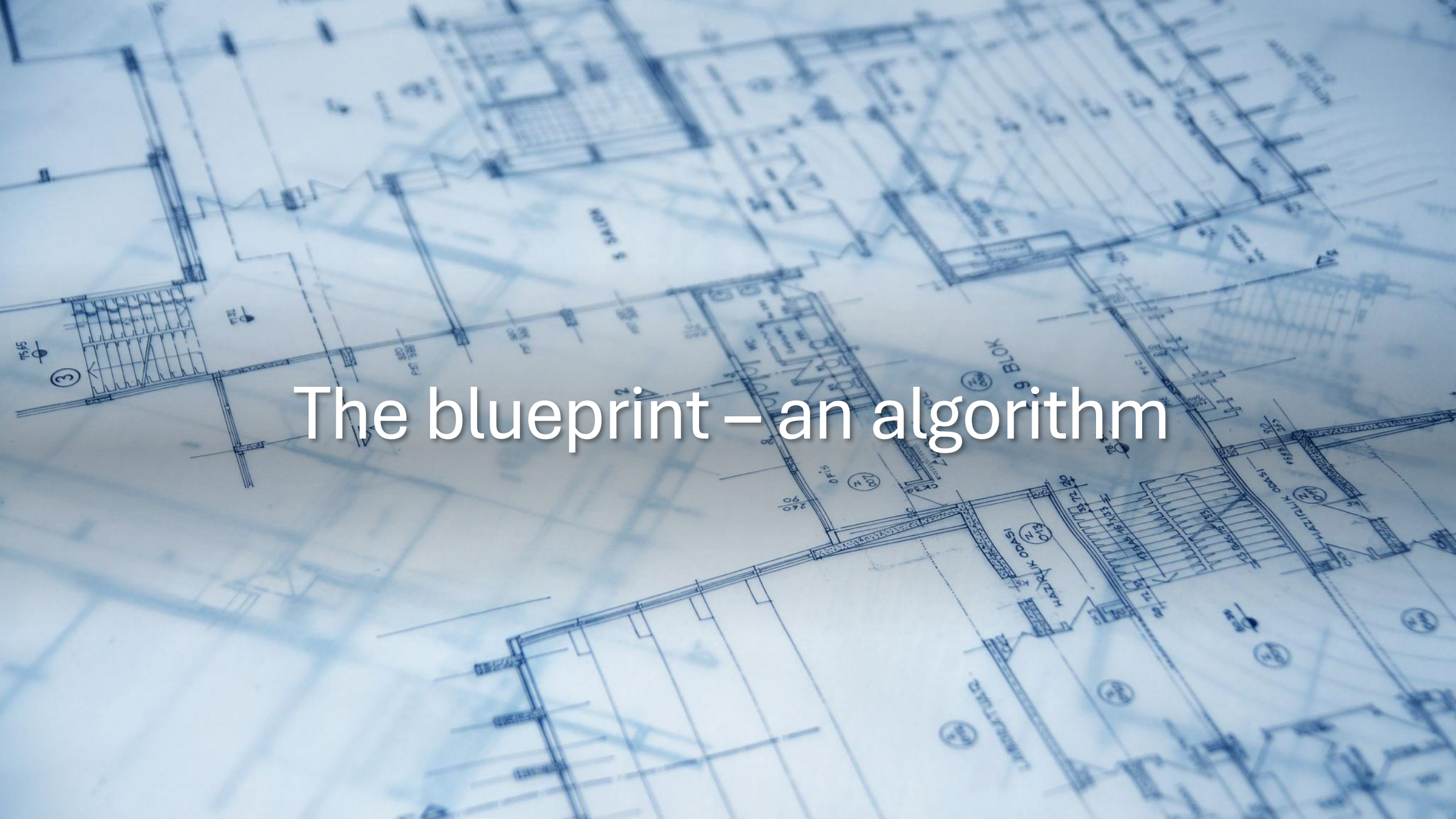


University of Szeged

Robert Nagy

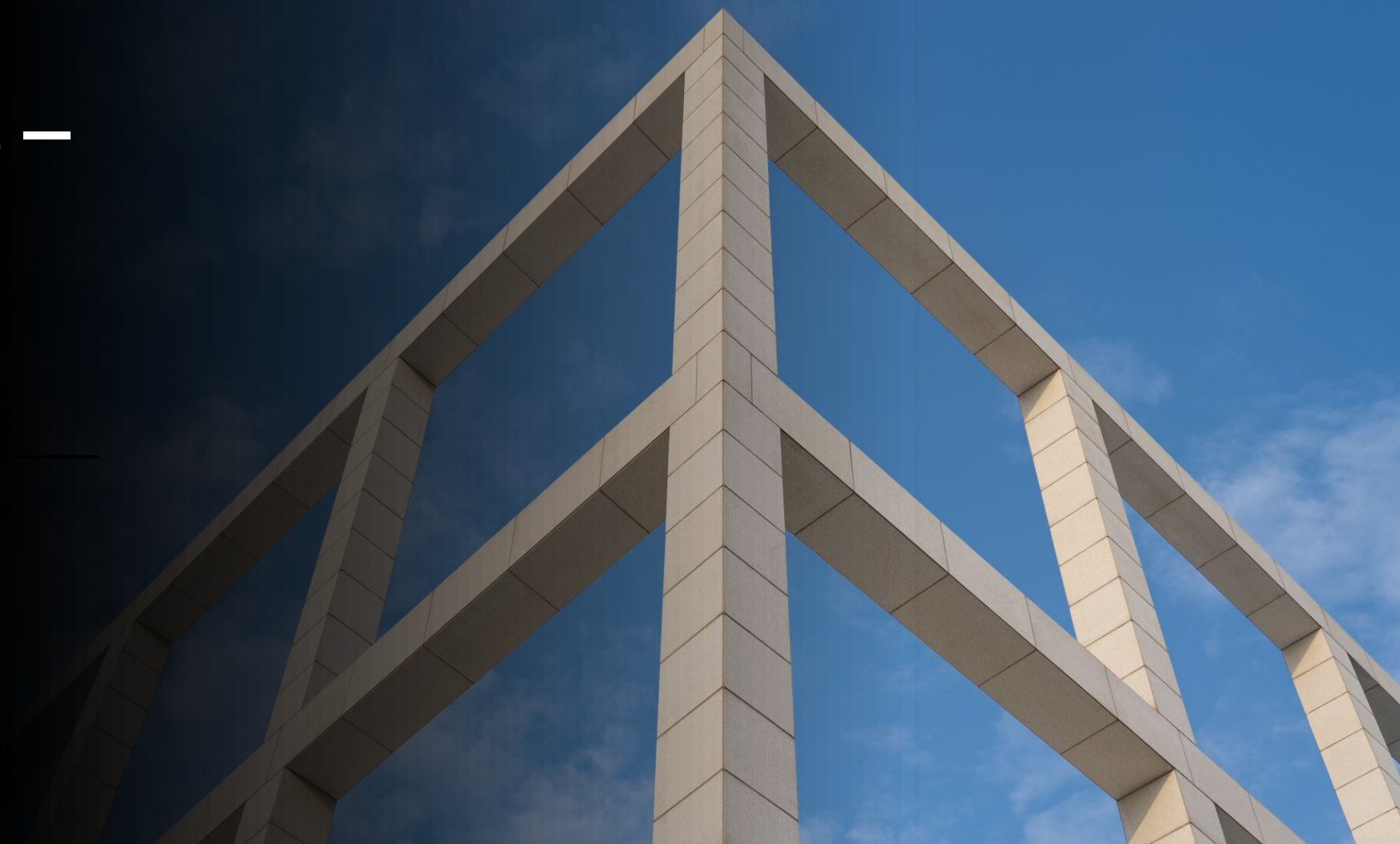
2025/26. Spring Semester


Imagine that you’re building a house



The blueprint – an algorithm

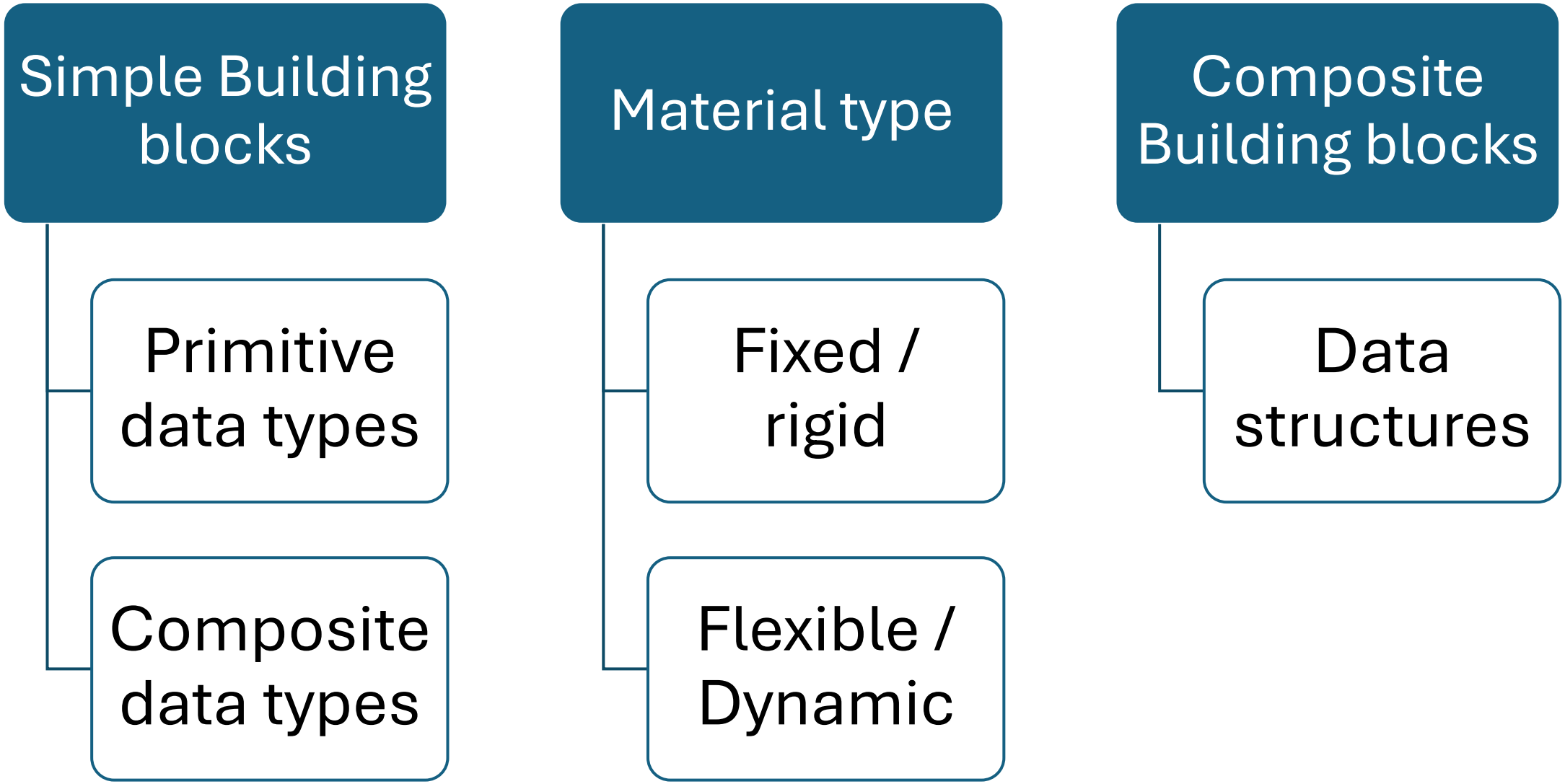
The materials – provide the structure



Even if the **blueprint** is **flawless**, *low-quality materials* can make the *house collapse* early.

Data - the building blocks for software




Simple Building blocks
Primitive data types
Composite data types
Material type
Fixed / rigid
Flexible / Dynamic
Composite Building blocks
Data structures

Primitive data types – fixed size in memory

- Int
- Float, Double
- Char
- Boolean
- Pointer

https://www.geeksforgeeks.org/dsa/data-types-in-programming/

Composite data types – fixed size in memory

- Objects (Class instances)
- ANSI C specific:
  - Struct
  - Union

Material types

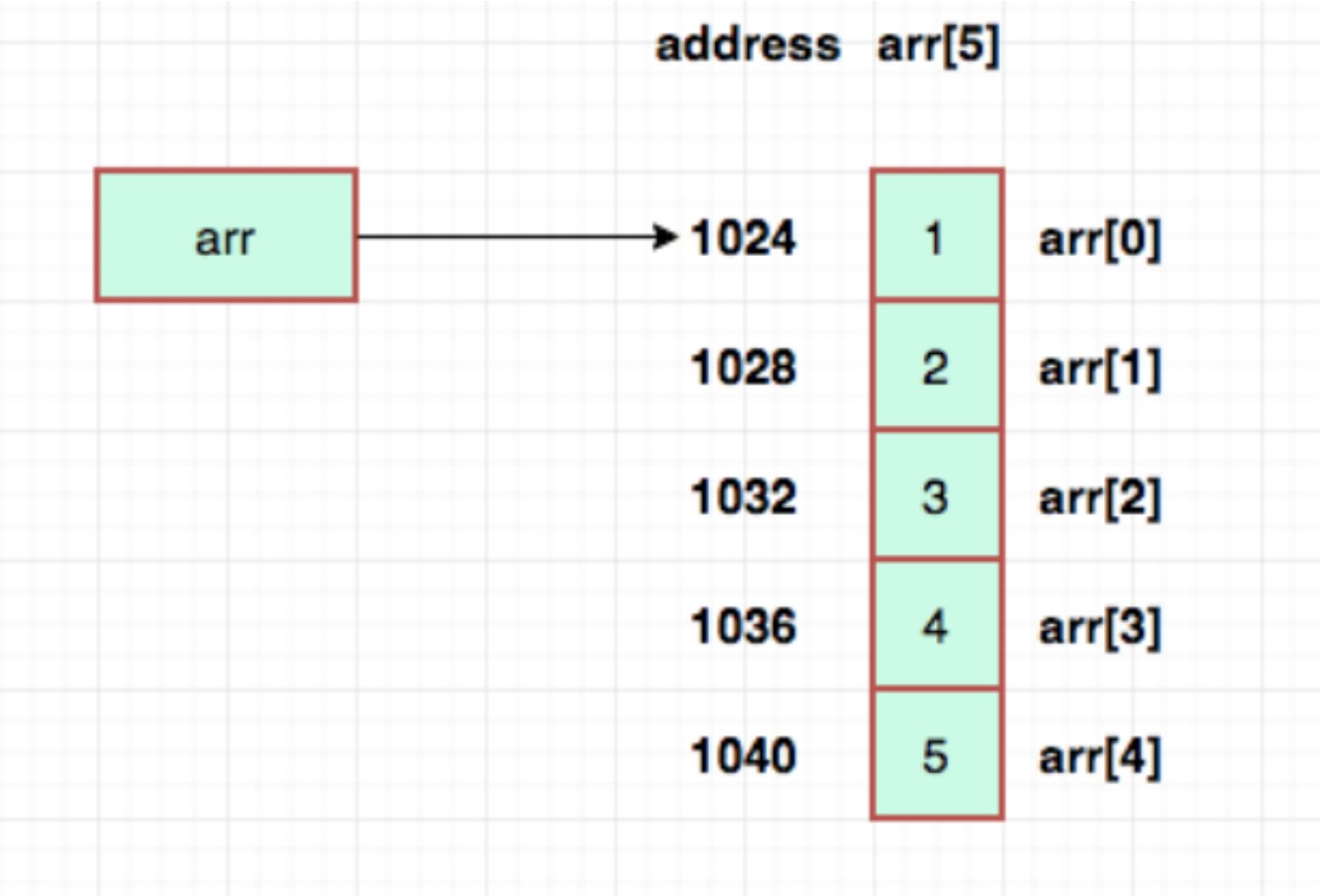
Rigid

- Fixed, contiguous place in memory

Flexible

- Usually a chain of Nodes
- Nodes have a fixed size
- The node-chain can grow indefinitely

Array – contiguous memory addresses

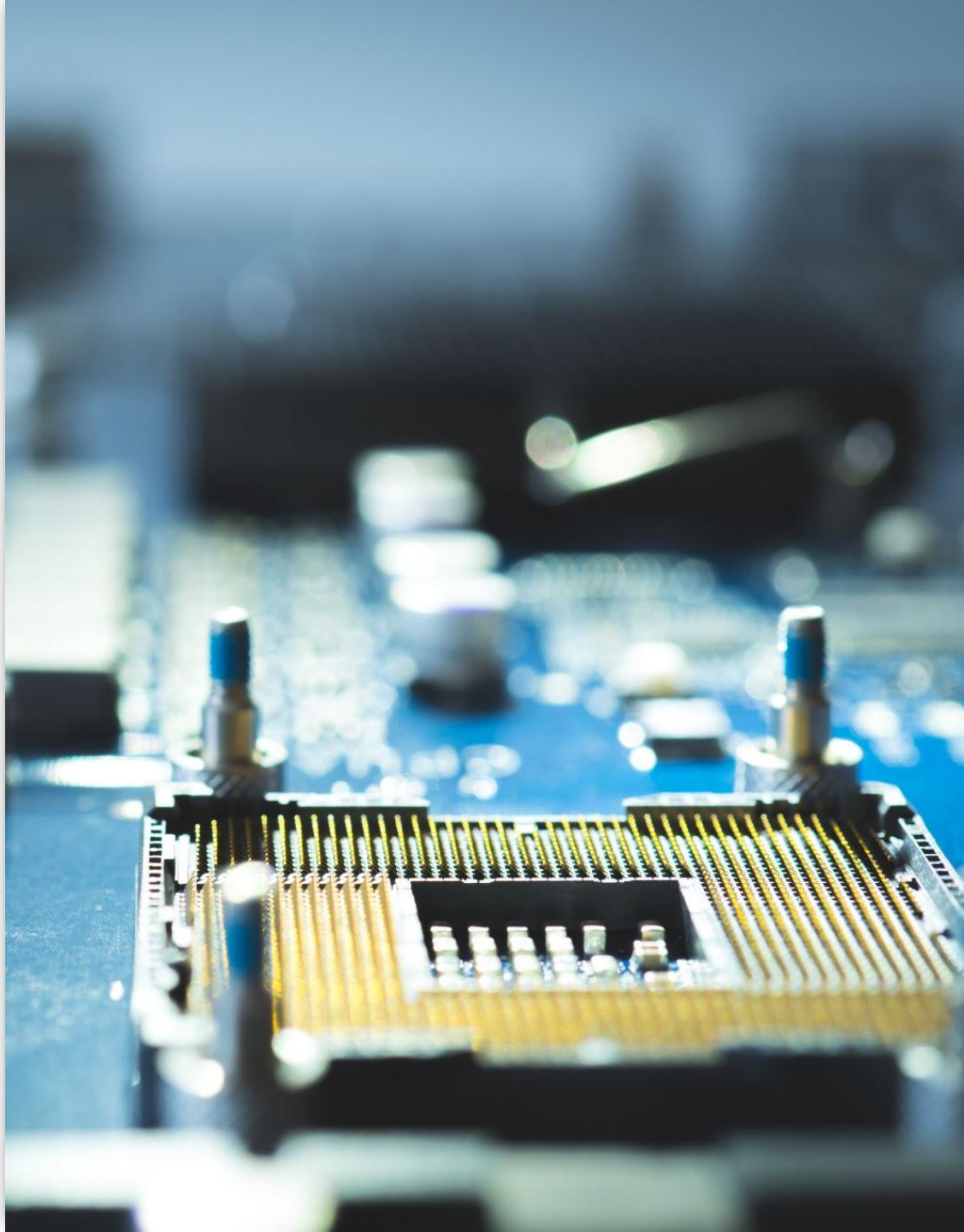


- Array that stores 32-bit integers

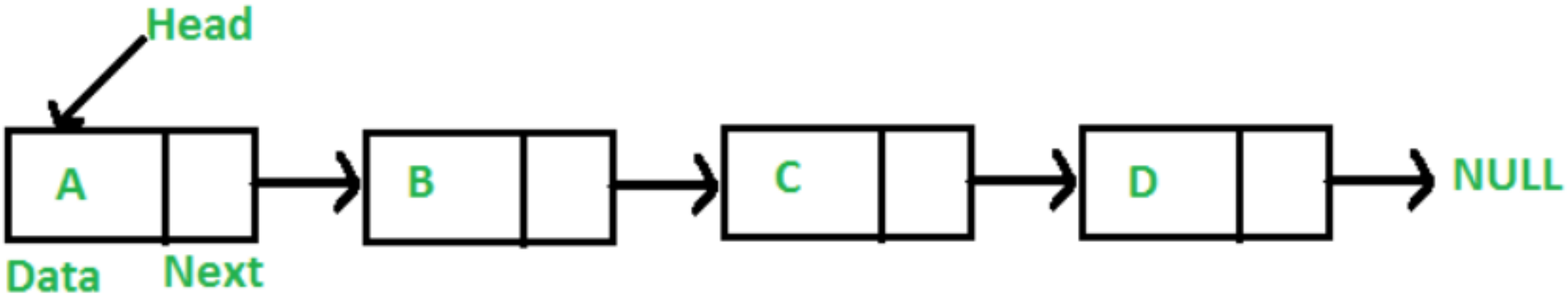
https://sites.duke.edu/writing270_02_ss12021_ecyoung/2021/06/28/data-structures-in-our-everyday-lives-arrays/

Pointer

- A variable that stores a memory address
- A memory address is just a number



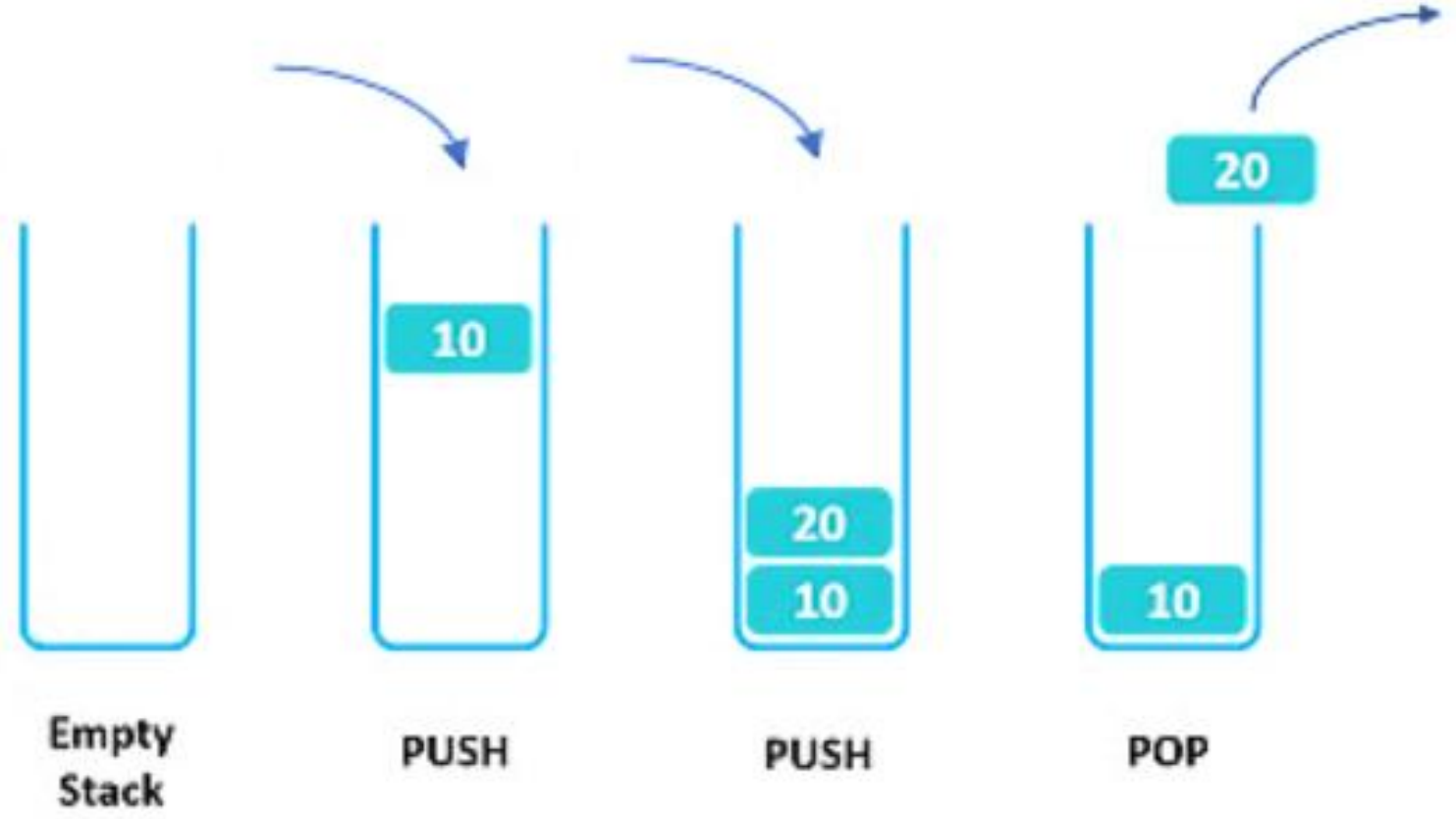
Nodes with pointers to other nodes



Data Structures

The Stack - LIFO





Usage - Examples

- Recursion –
  - Transforming recursive code into iterative code
  - Call stack

Usage - Examples

- Recursion –
  - Transforming recursive code into iterative code
  - Call stack
- Backtracking algorithms
  - Maze solvers

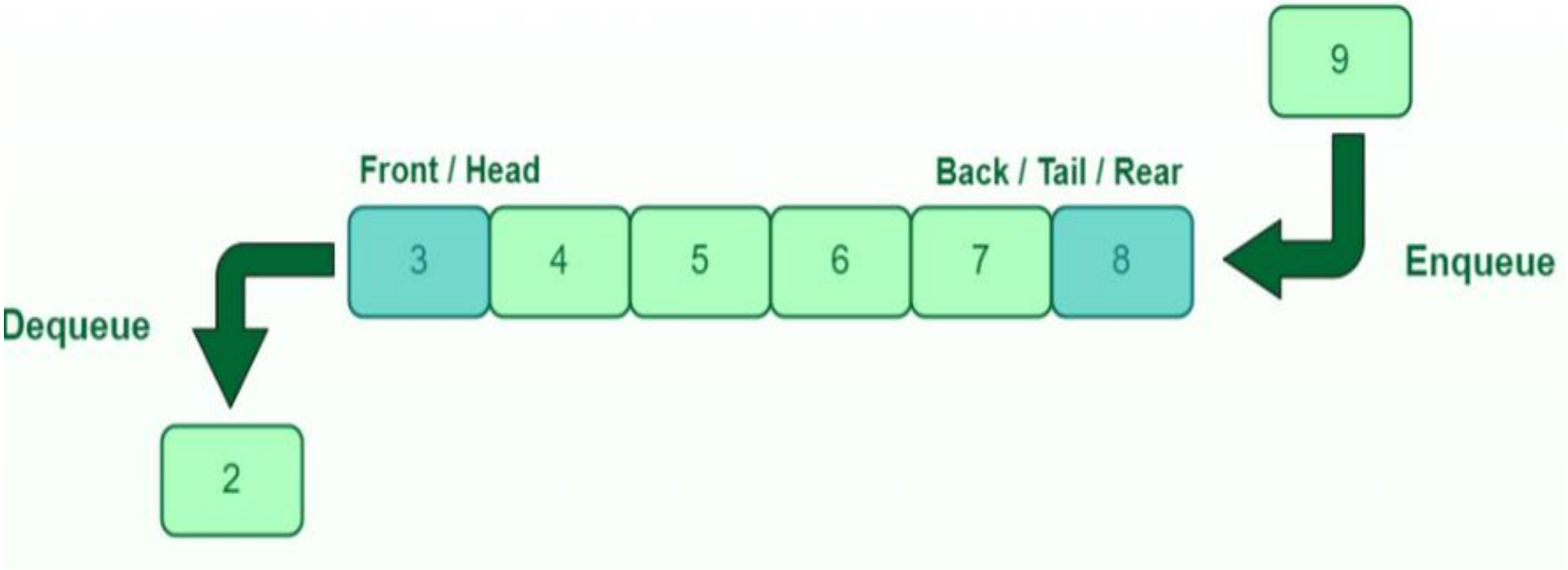
Usage - Examples

- Recursion –
  - Transforming recursive code into iterative code
  - Call stack
- Backtracking algorithms
  - Maze solvers
- Undo/Redo operation

Usage - Examples

- Recursion –
  - Transforming recursive code into iterative code
  - Call stack
- Backtracking algorithms
  - Maze solvers
- Undo/Redo operation
- Navigation in web browsers and web apps

The Queue - FIFO



Usage - Examples

- Operating systems
  - Task scheduling

Usage - Examples

- Operating systems
  - Task scheduling
- Network routers and switches
  - Data packet management

Usage - Examples

- Operating systems
  - Task scheduling
- Network routers and switches
  - Data packet management
- Printers
- Customer service systems

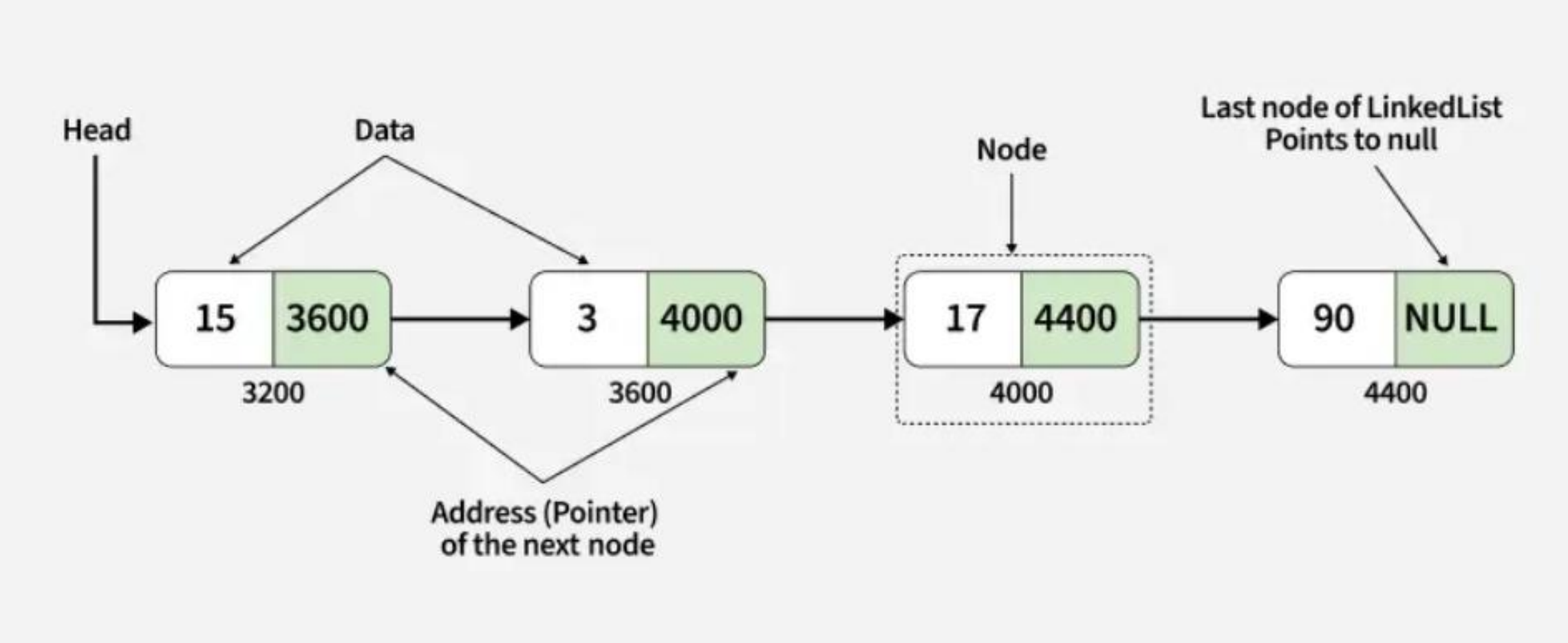
Exercises



The following data stream arrives: 1, 6, 8, 13, 42, 7, 4

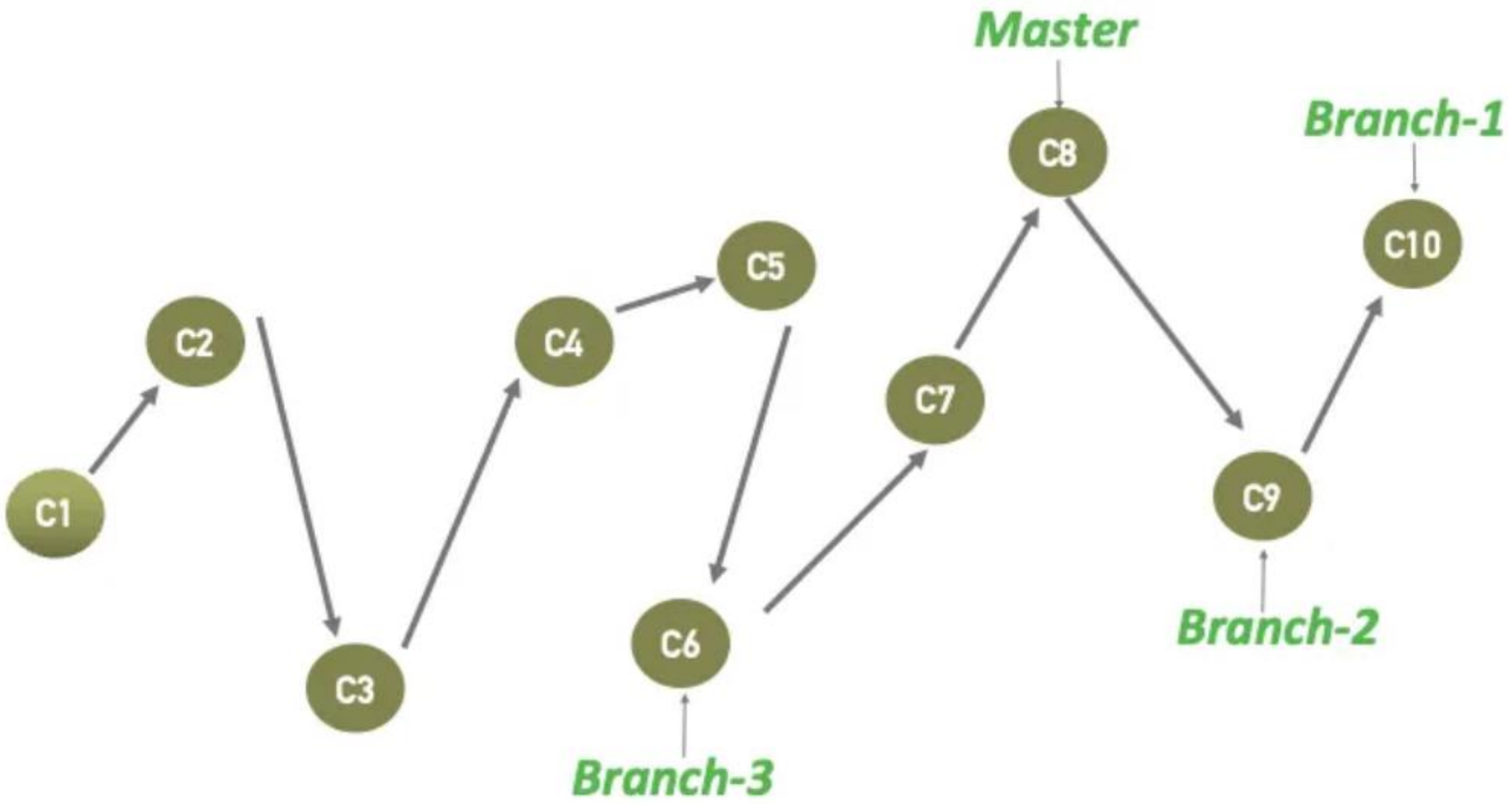
1. Build a queue, starting with the first element, moving left to right
2. Dequeue all the elements one by one, and push them onto a stack
3. Pop all elements from the stack

Linked List

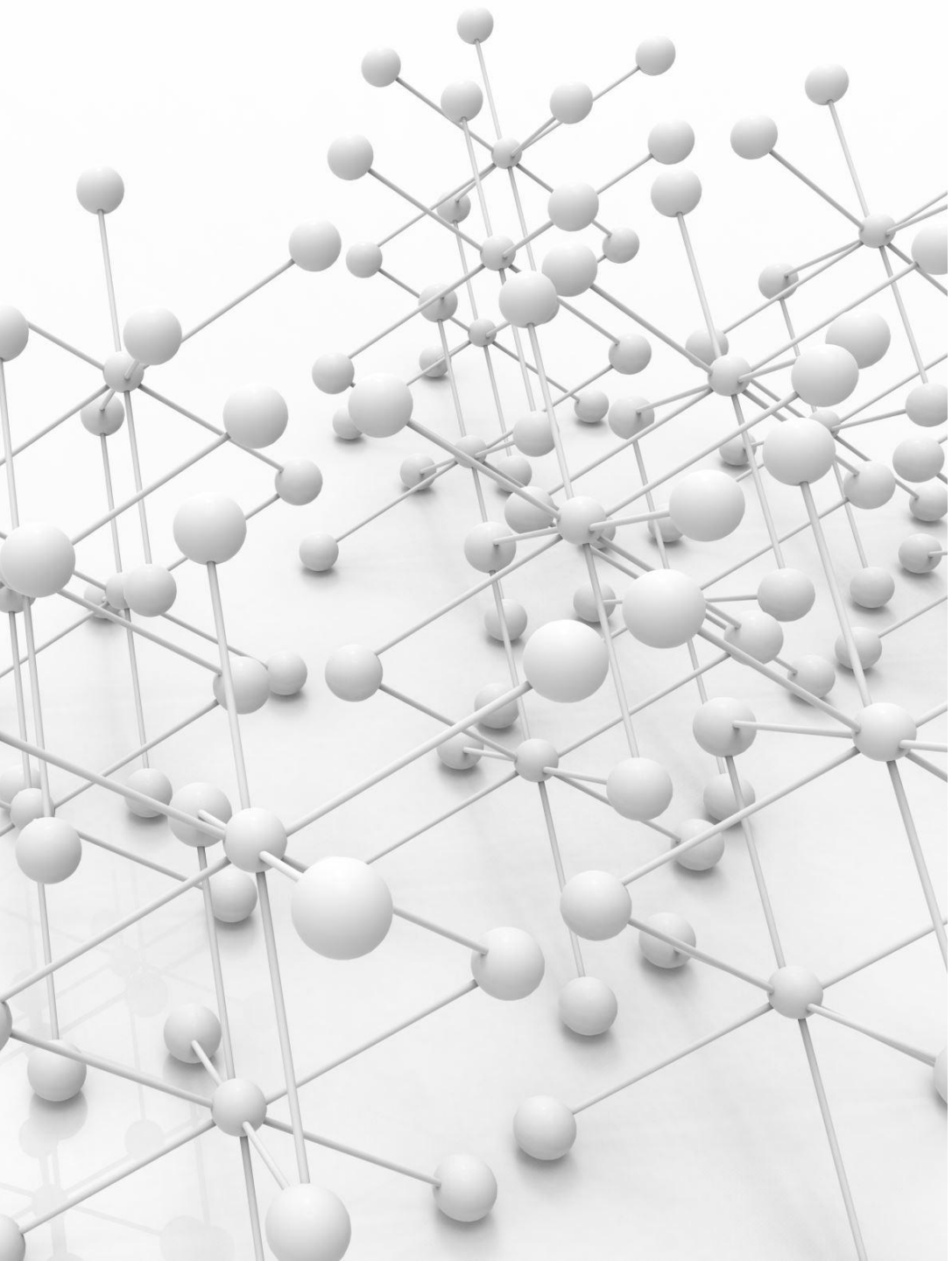

Head
Data
Node
Last node of LinkedList
Points to null
15
3600
3
4000
17
4400
90
NULL
3200
3600
4000
4400
Address (Pointer)
of the next node

https://medium.com/@vivektikar/demystifying-git-what-is-a-git-branch-exactly-under-the-hood-b0a607ba1129

Git



Pool of git commits



Binary heap

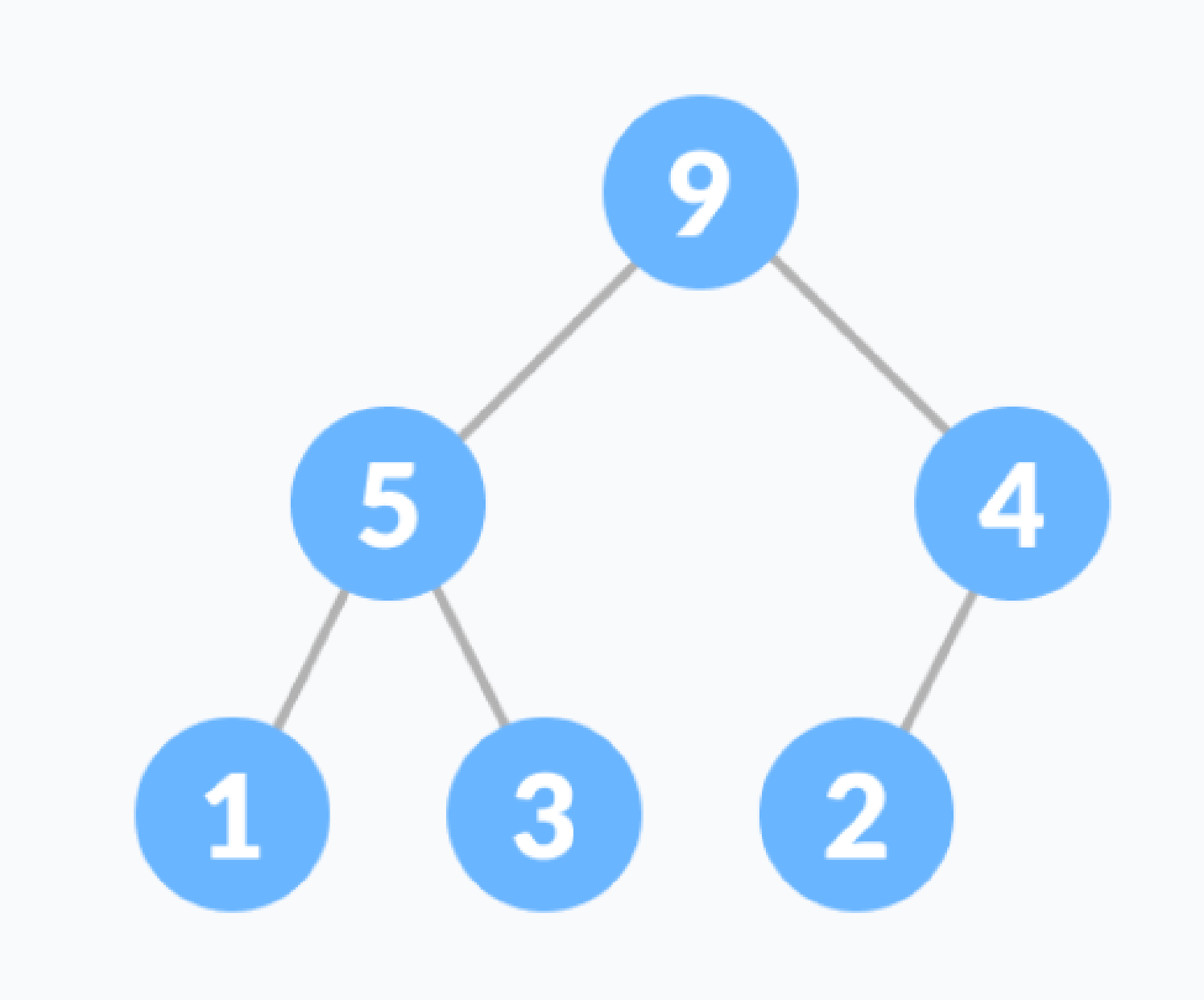
Read more: https://www.programiz.com/dsa/heap-data-structure

2 Types

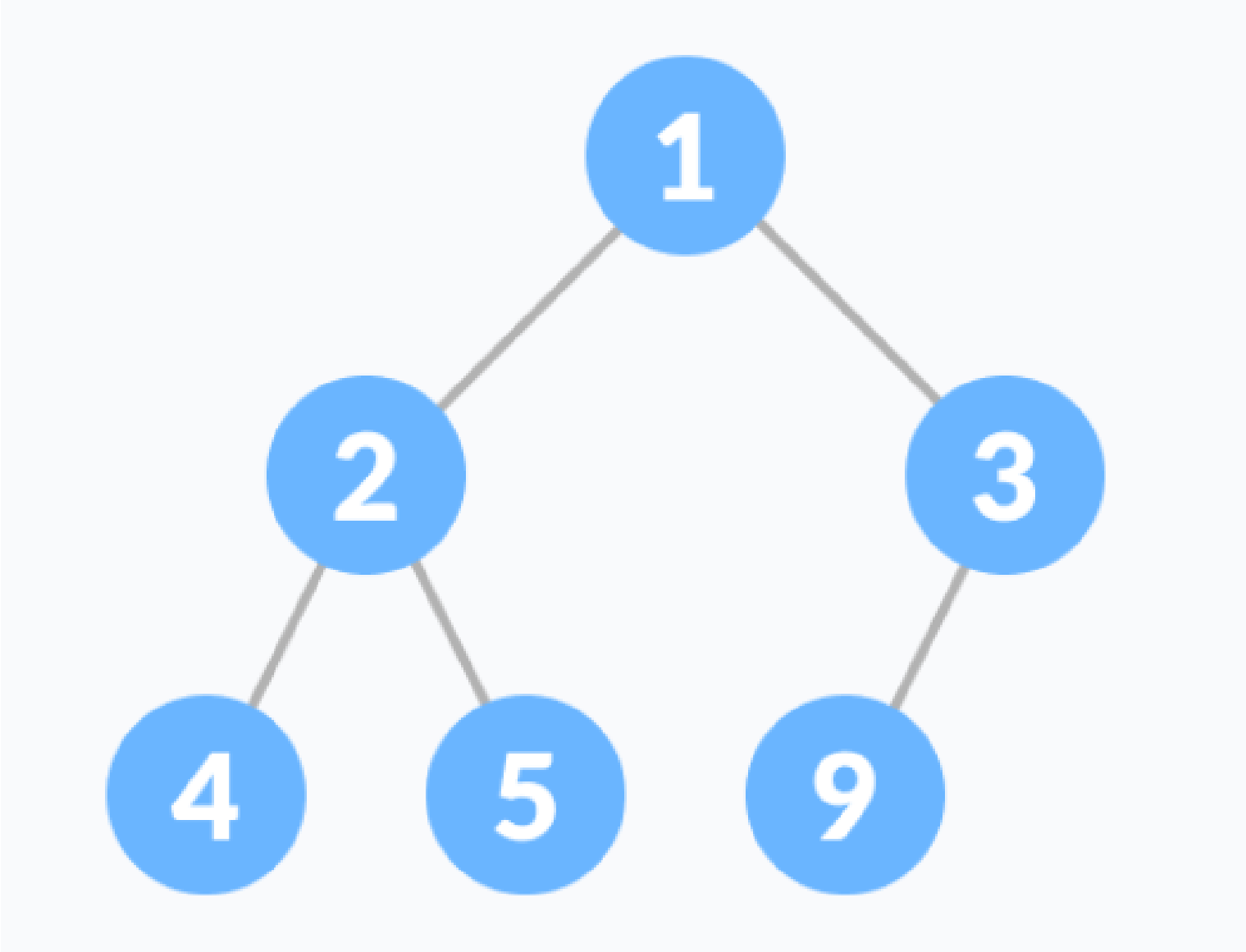
Min heap

Max heap

Max heap



Min heap



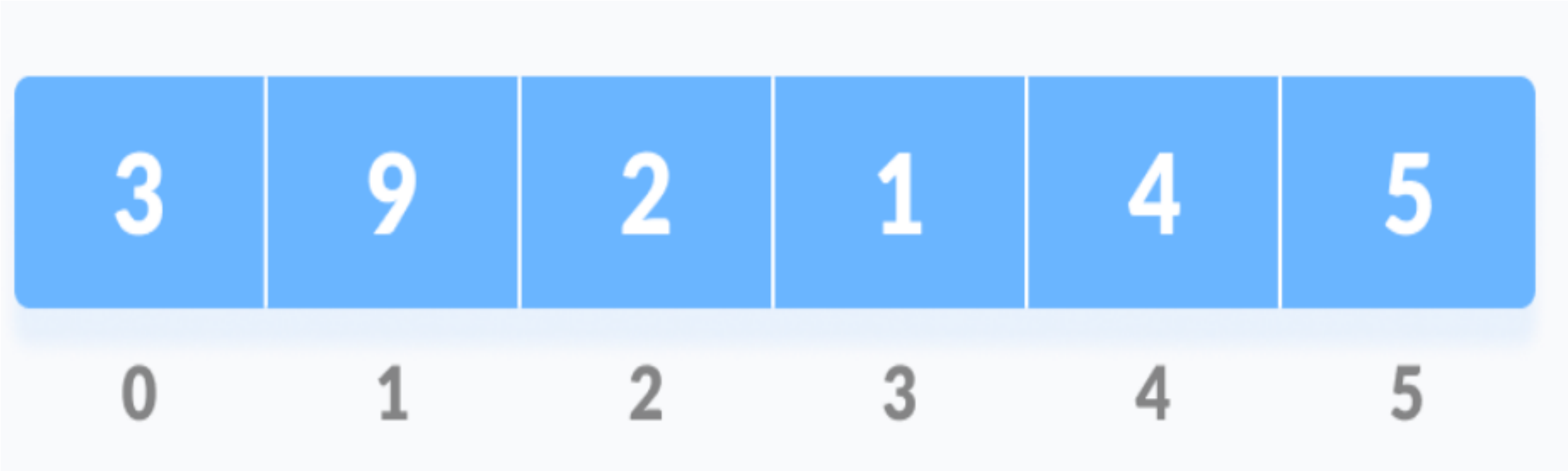
Heap properties & functions

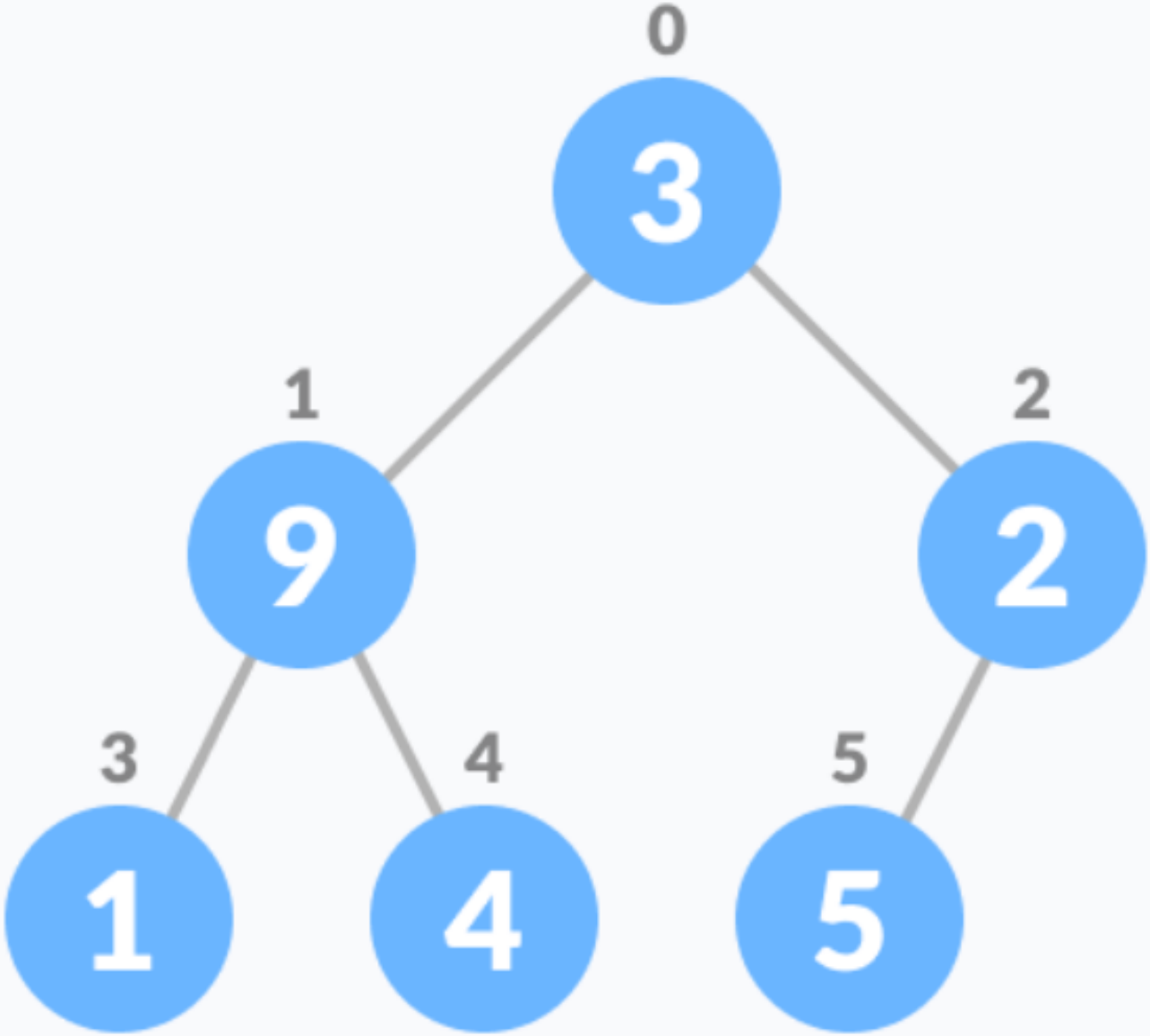
A heap is a **complete binary tree**

“Heapify” procedure

Heapify

Step 1: Input array





Complete Binary Tree

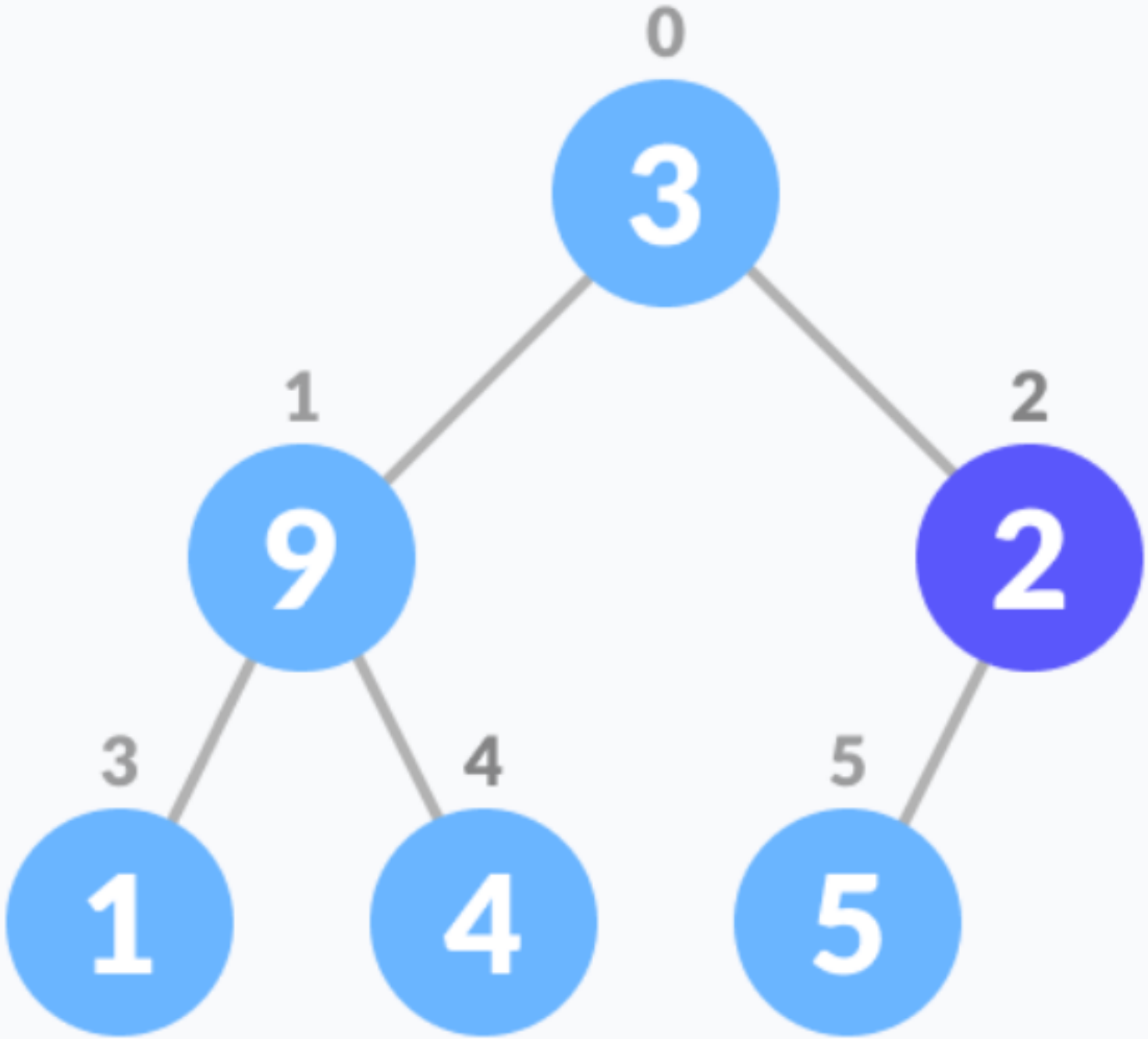
- Step 2 – Construct the CBT From the input array

How?

- The *left* child of the element at index k is at index $2k + 1$

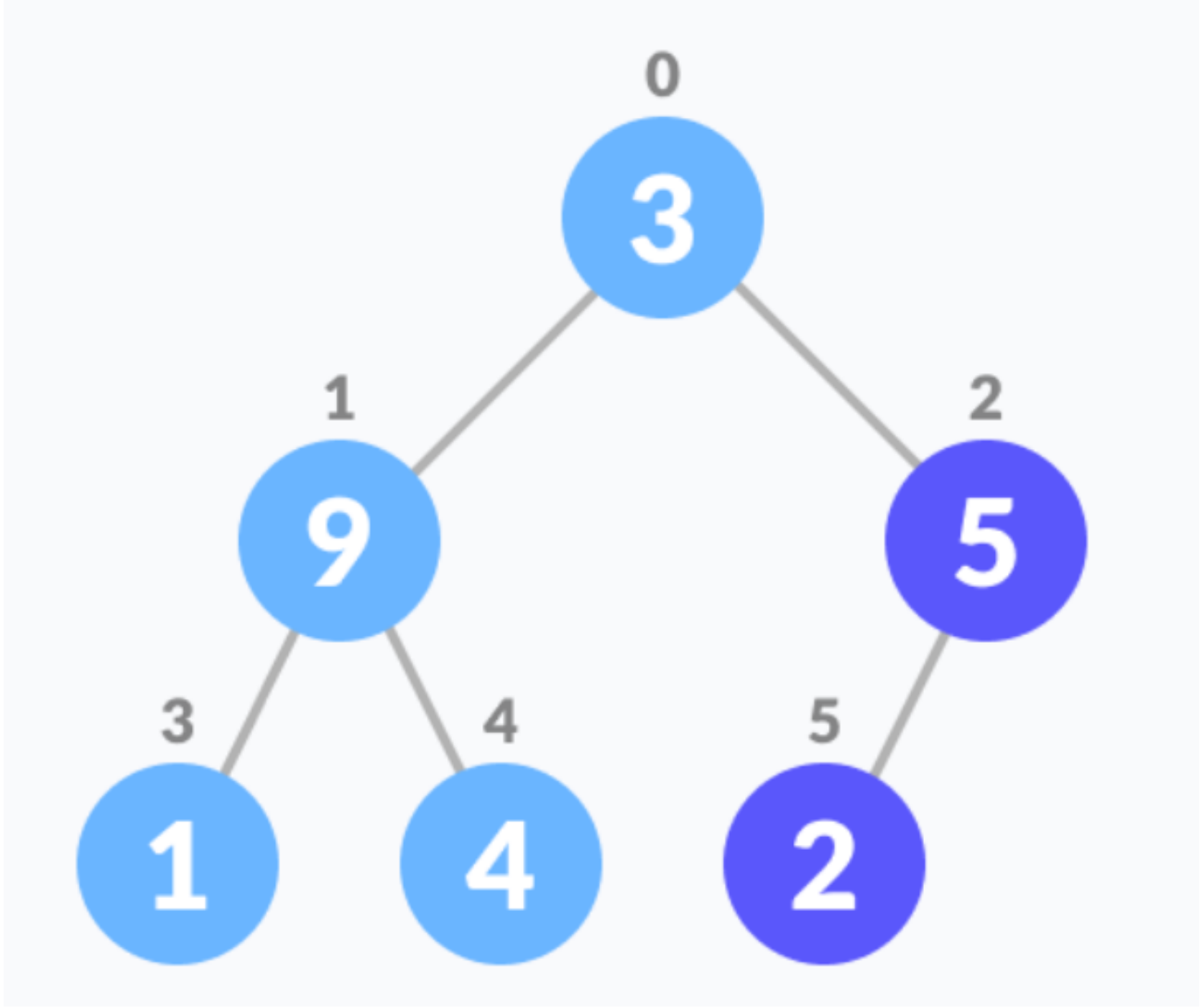
How?

- The *left* child of the element at index k is at index $2k + 1$
- The **right** child of the element at index k is at index $2k + 2$



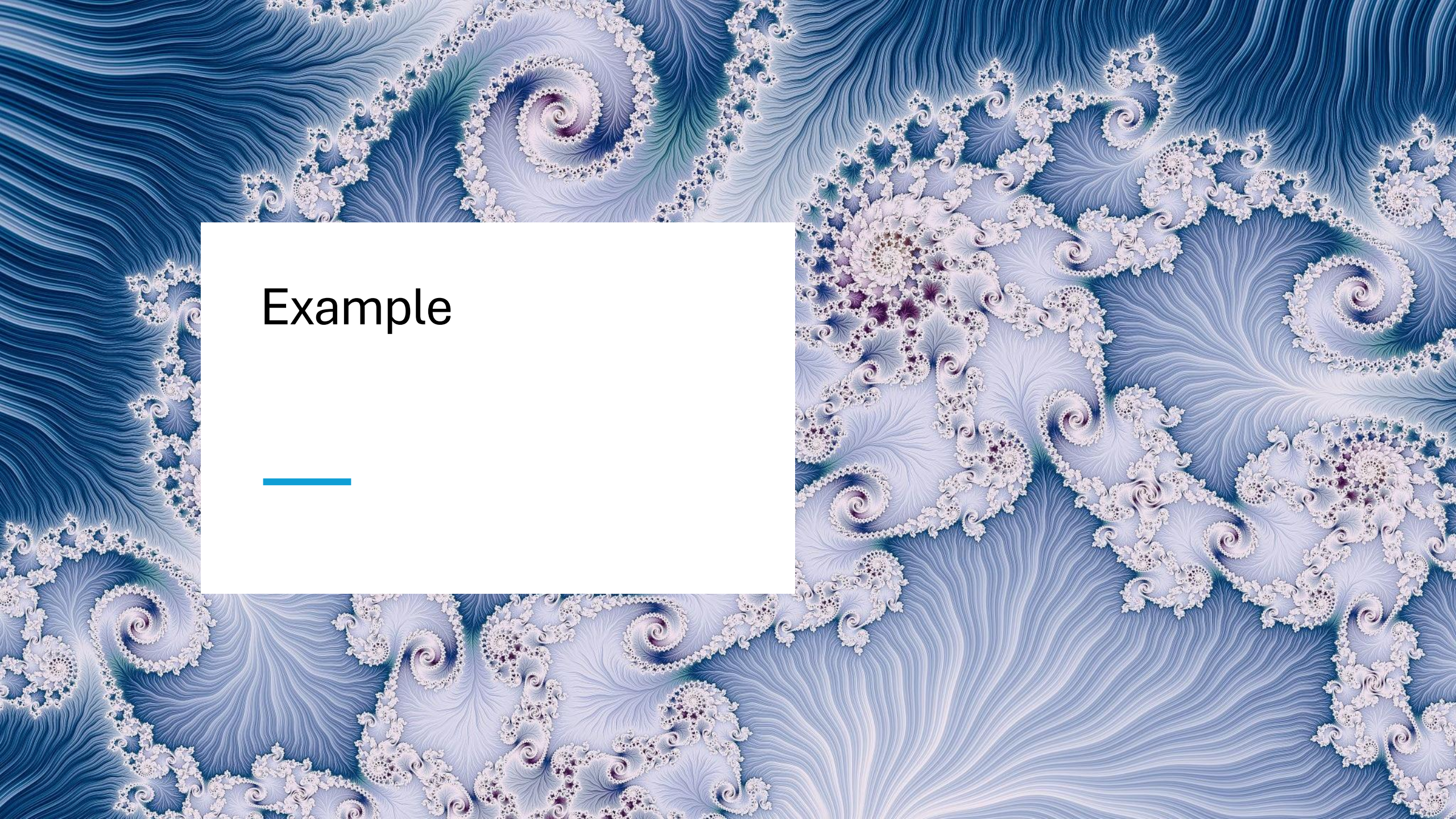
Step 3: “Heapify” – “Bubble up”

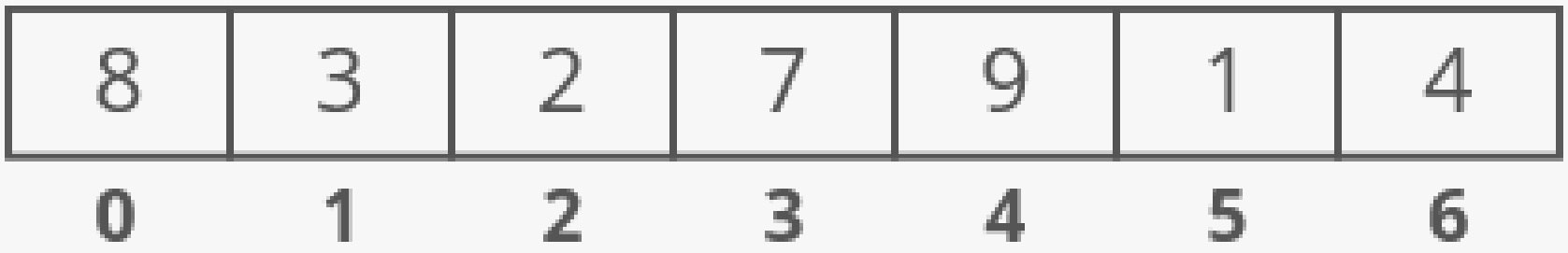
- Start from the first non-leaf node:
- index = $\frac{n}{2} - 1$

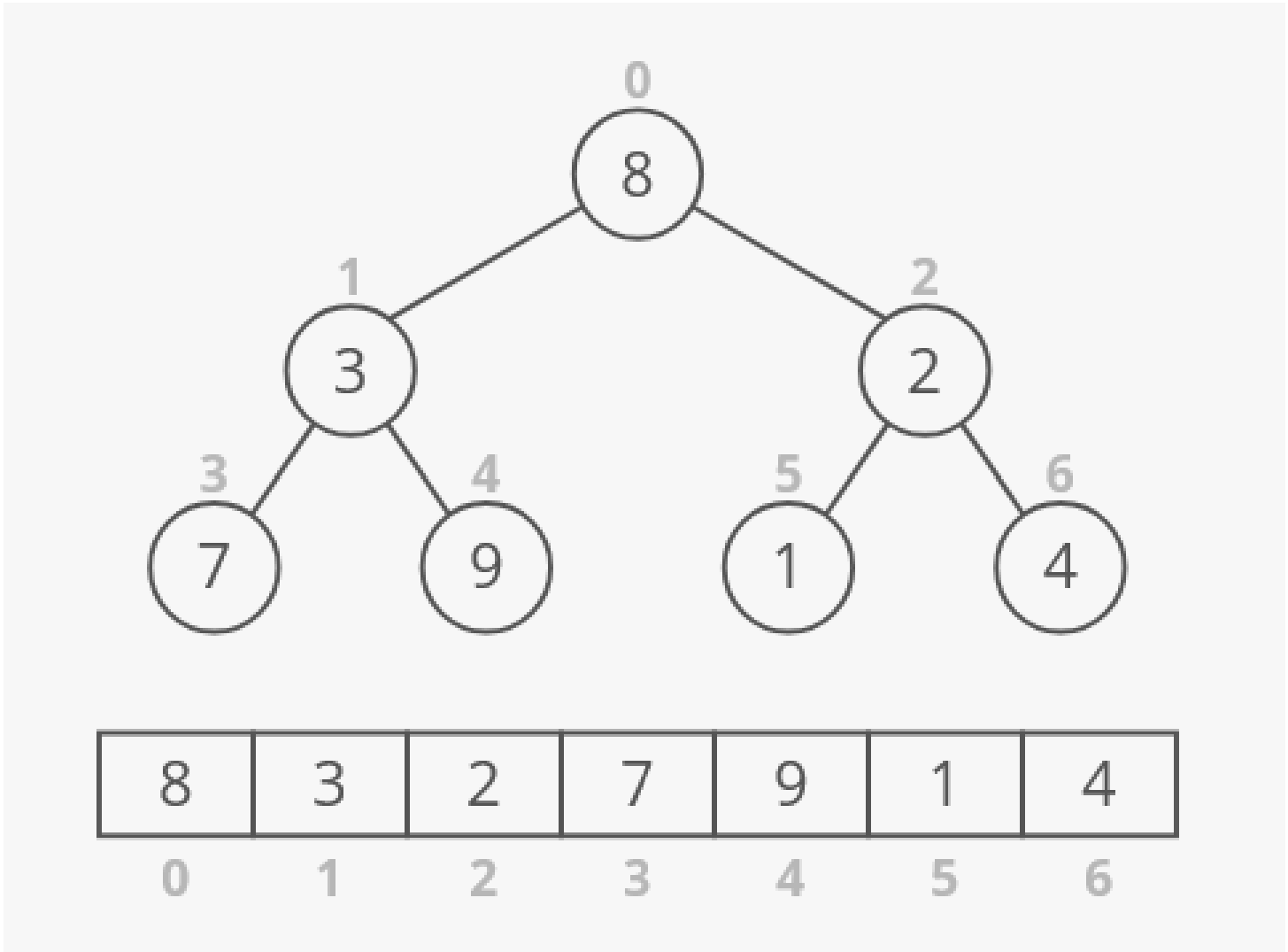


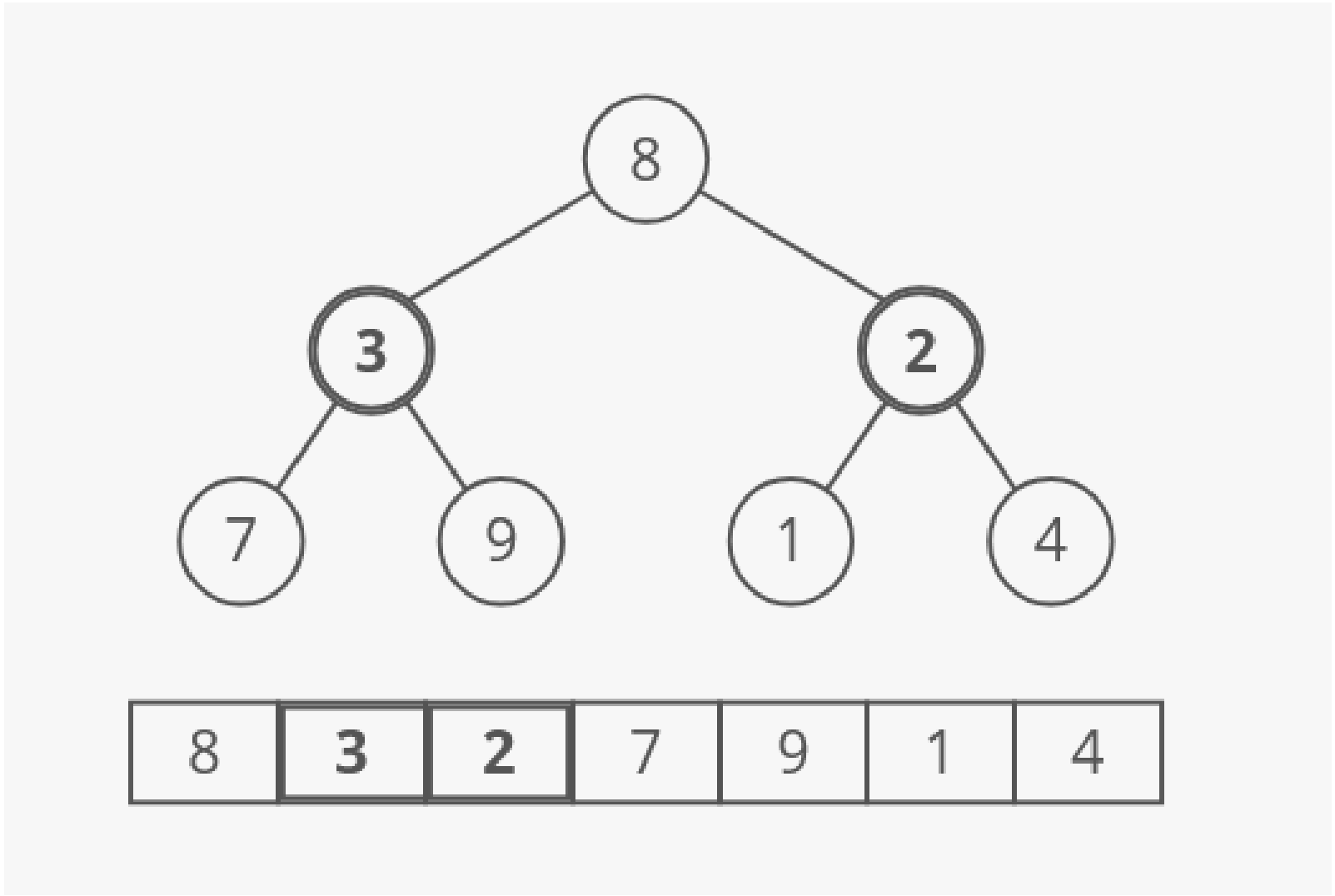
Swap the element with its biggest child

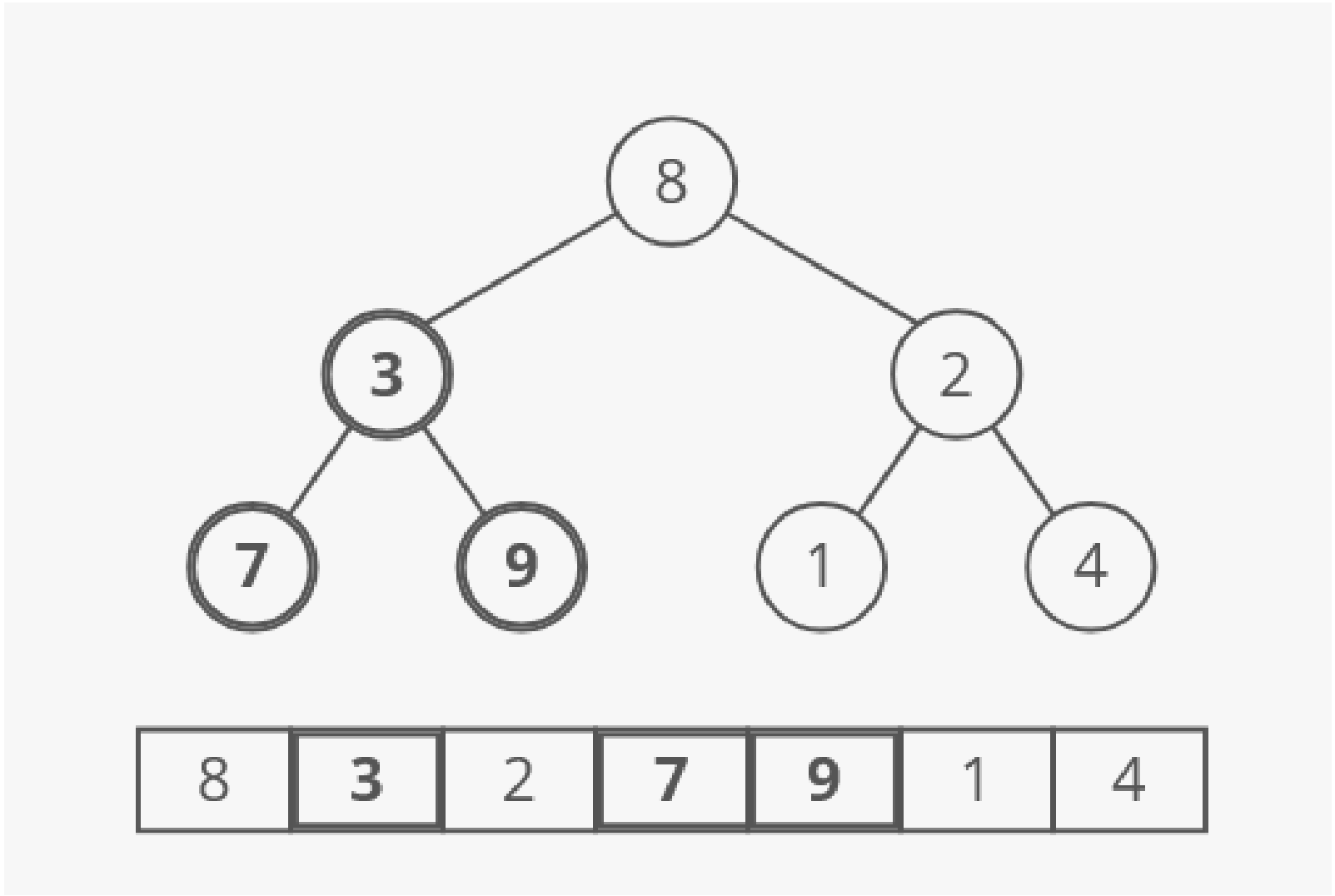
Repeat until done

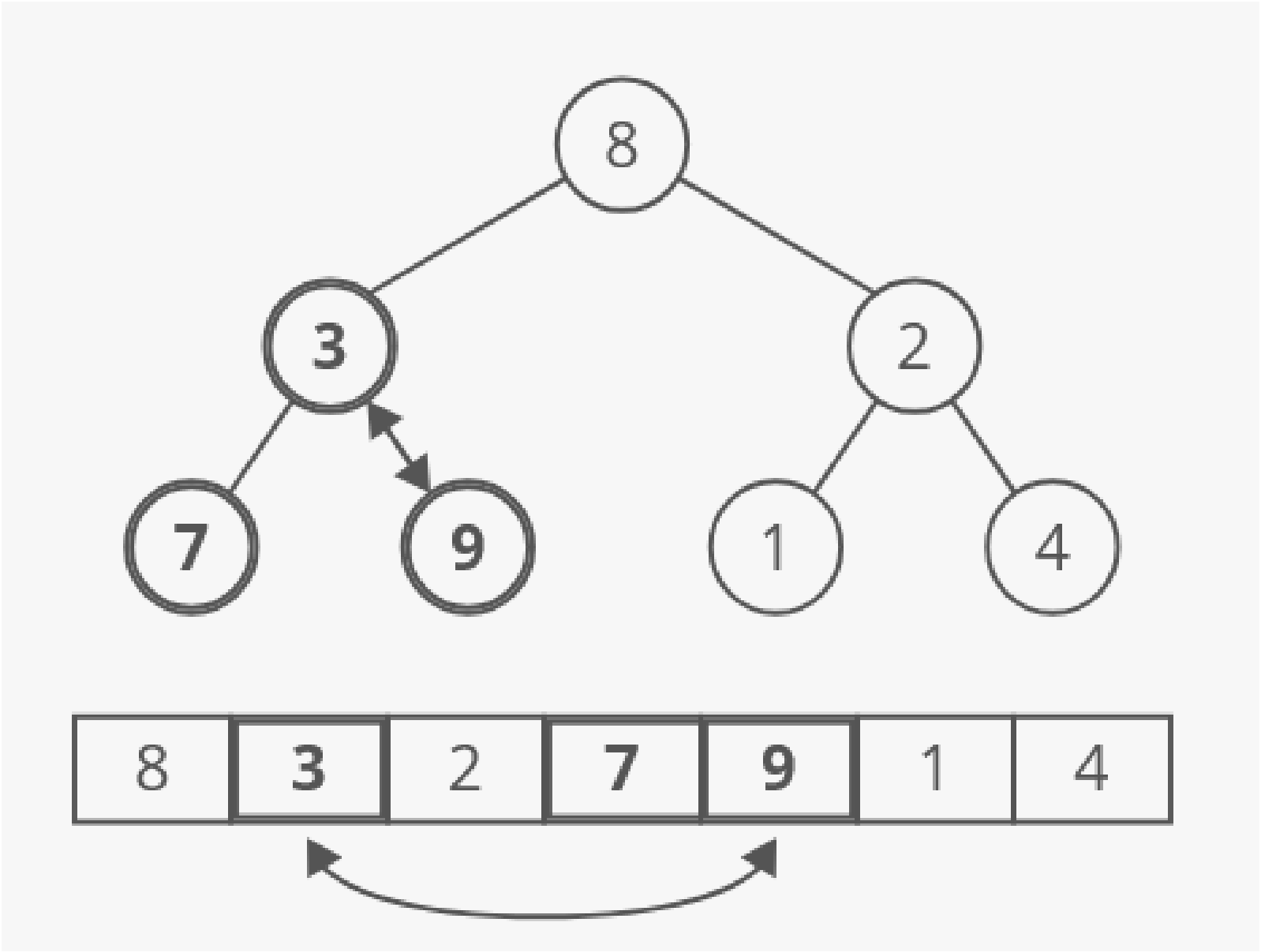

Example

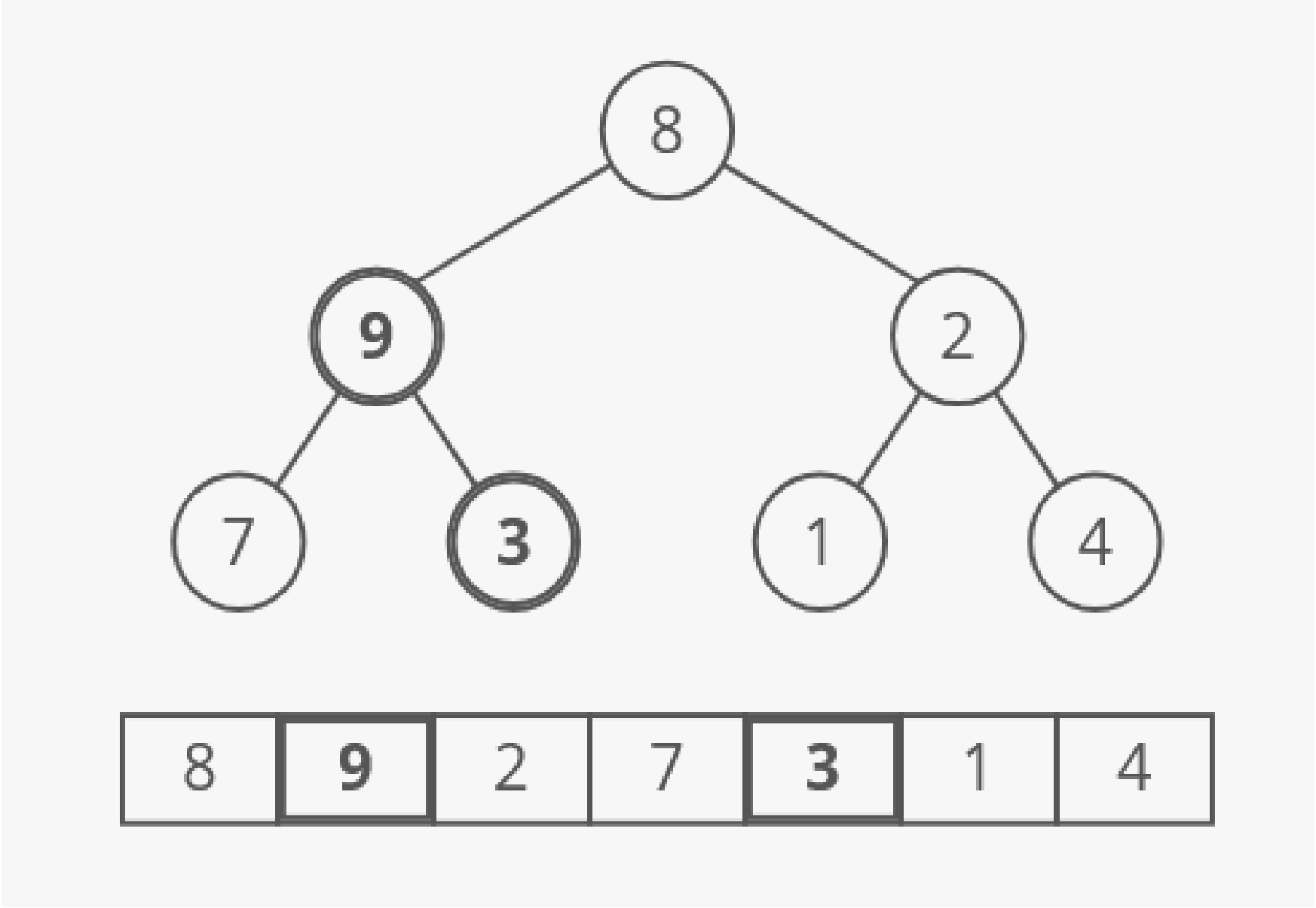

8
3
2
7
9
1
4
0
1
2
3
4
5
6

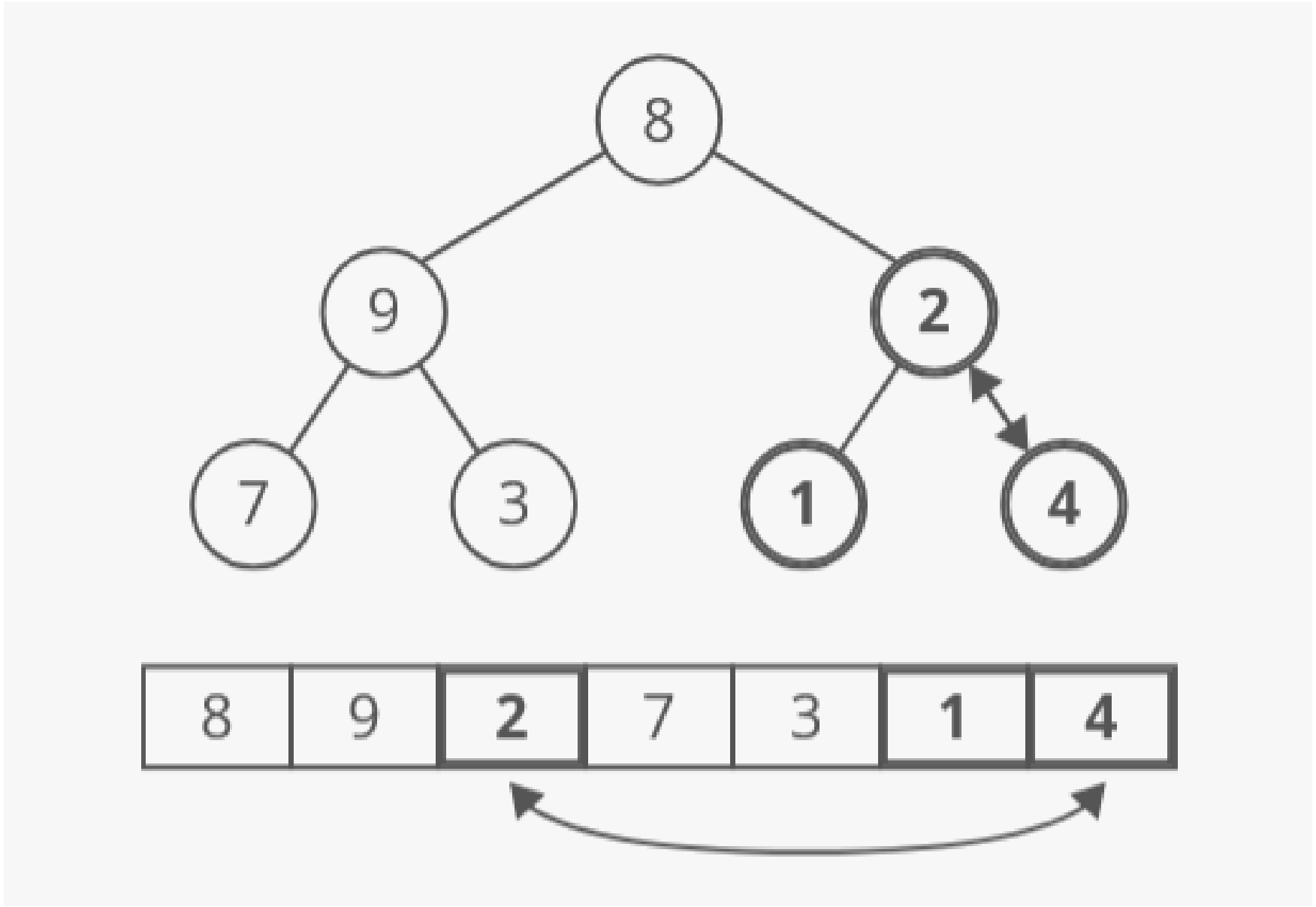

0
8
1
3
2
2
3
7
4
9
5
1
6
4
8
3
2
7
9
1
4
0
1
2
3
4
5
6

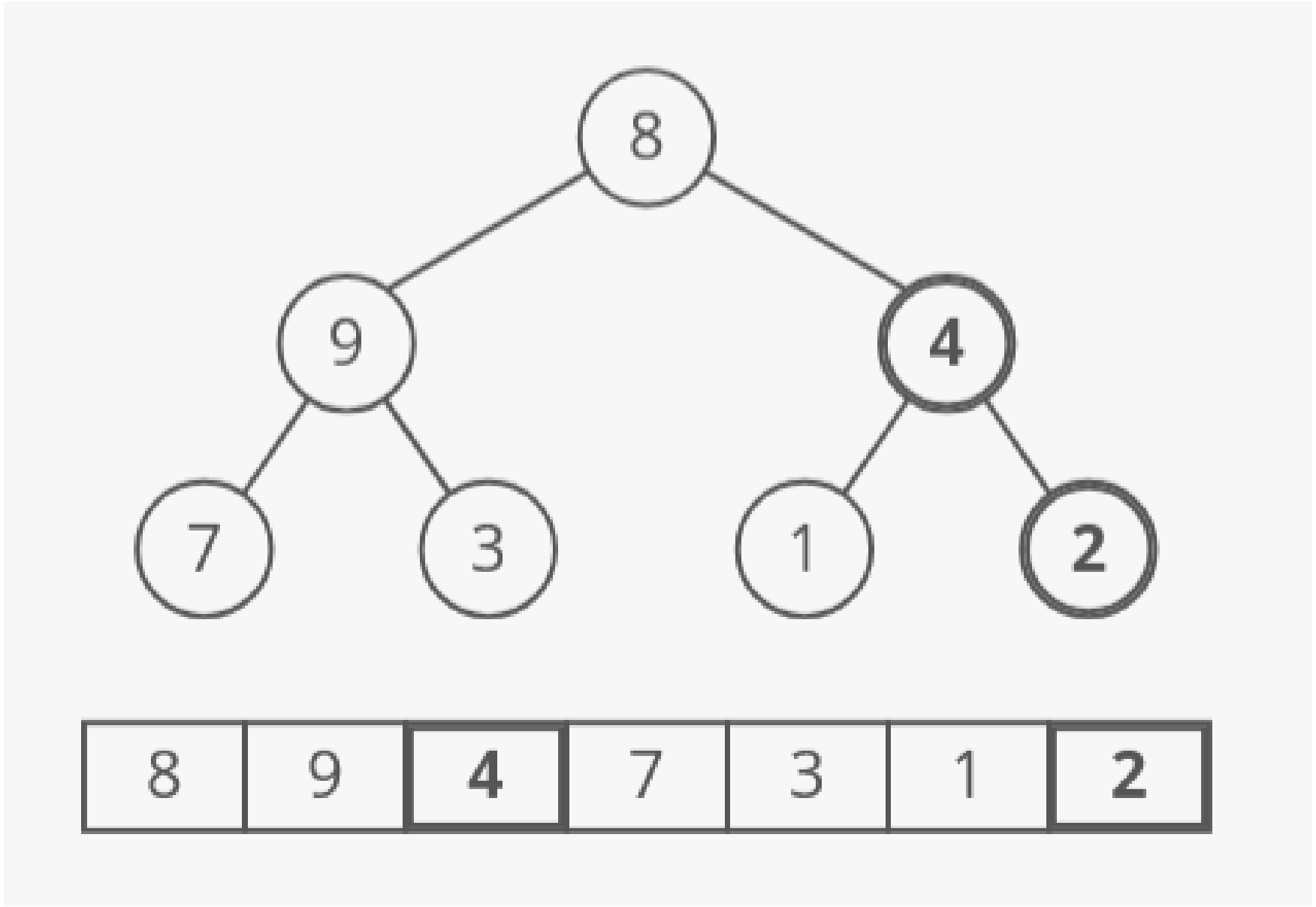

8
3
2
7
9
1
4
8
3
2
7
9
1
4

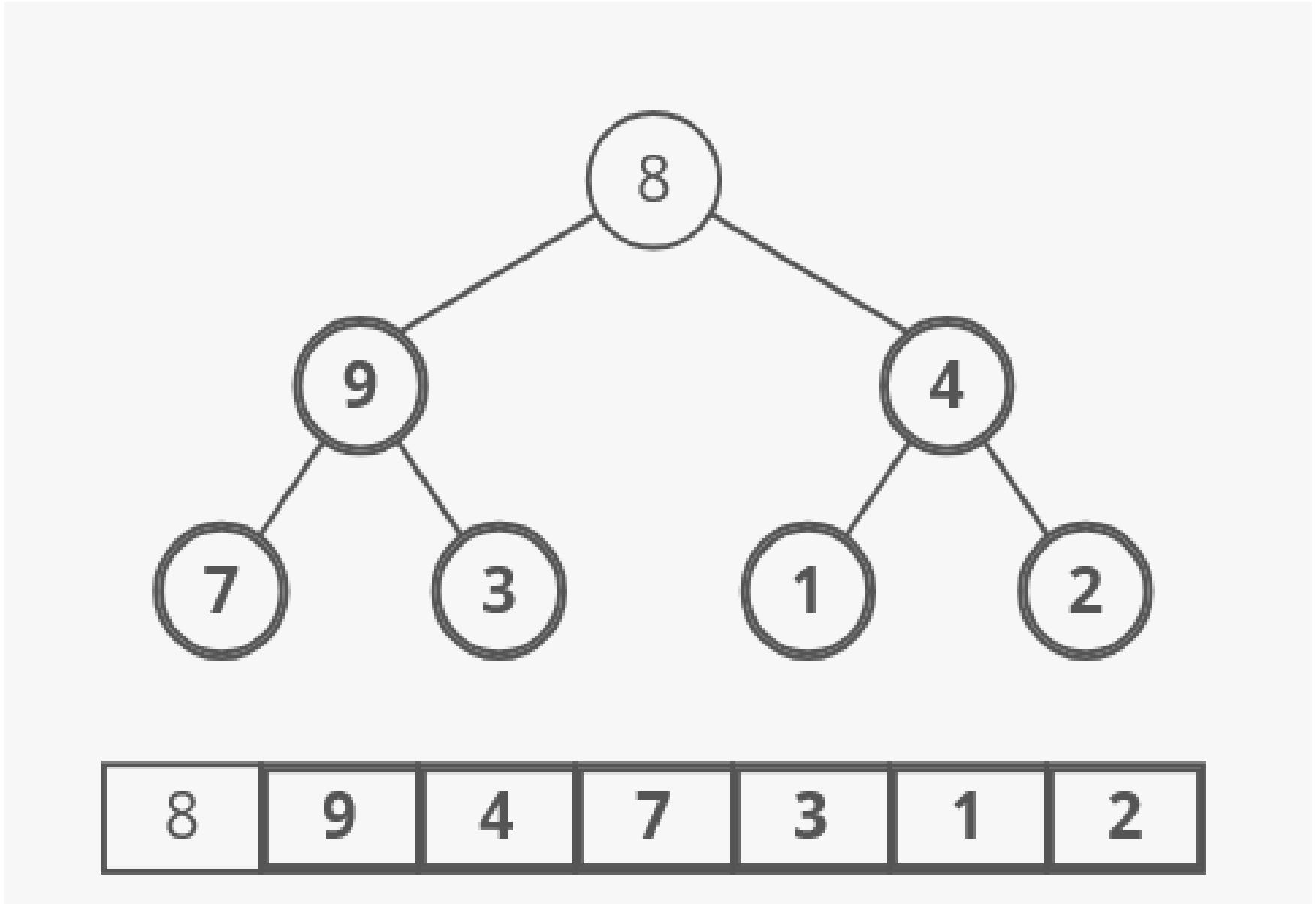

8
3
2
7
9
1
4
8
3
2
7
9
1
4

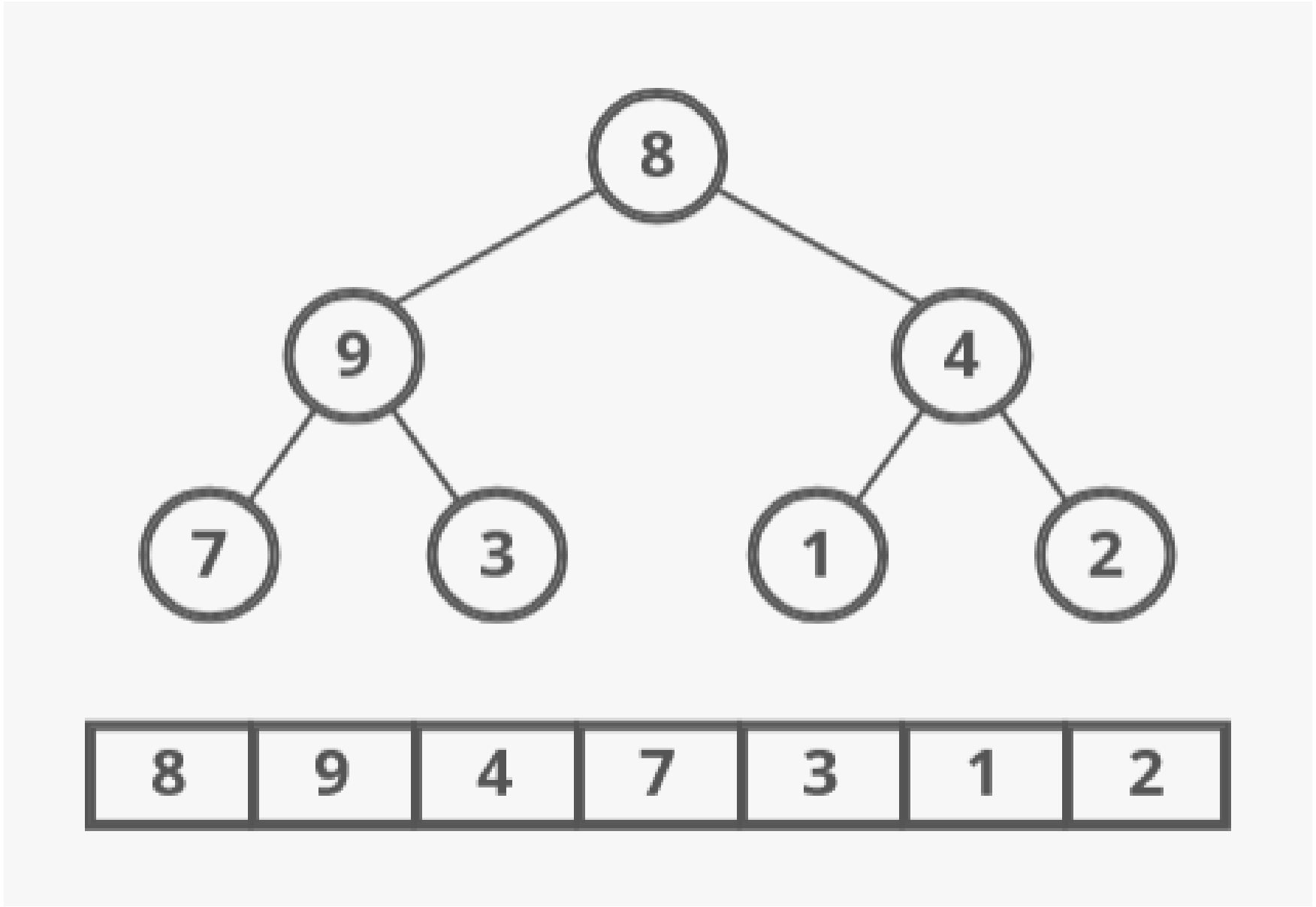

8
3
2
7
9
1
4
8
3
2
7
9
1
4

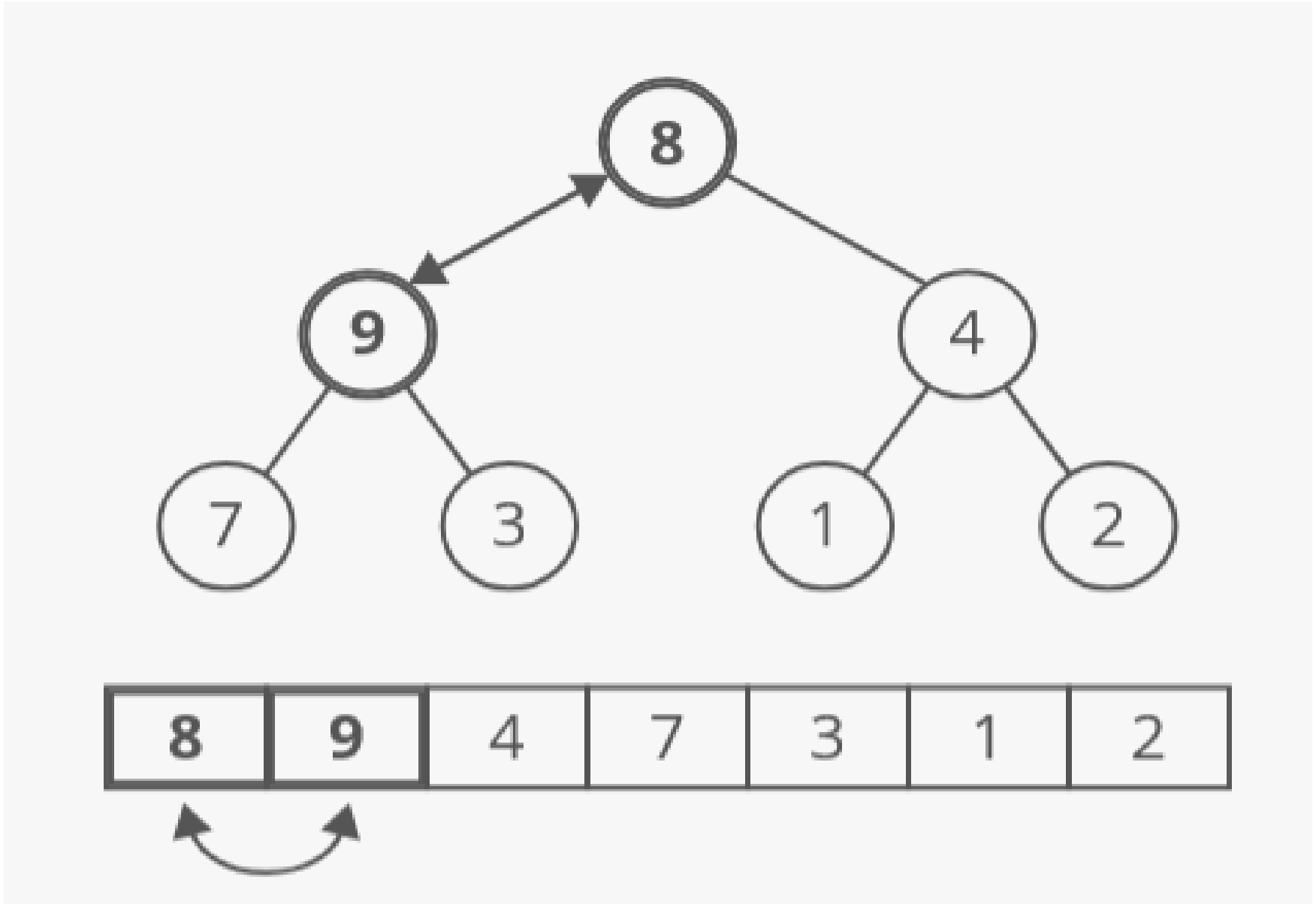

8
9
2
7
3
1
4
8
9
2
7
3
1
4

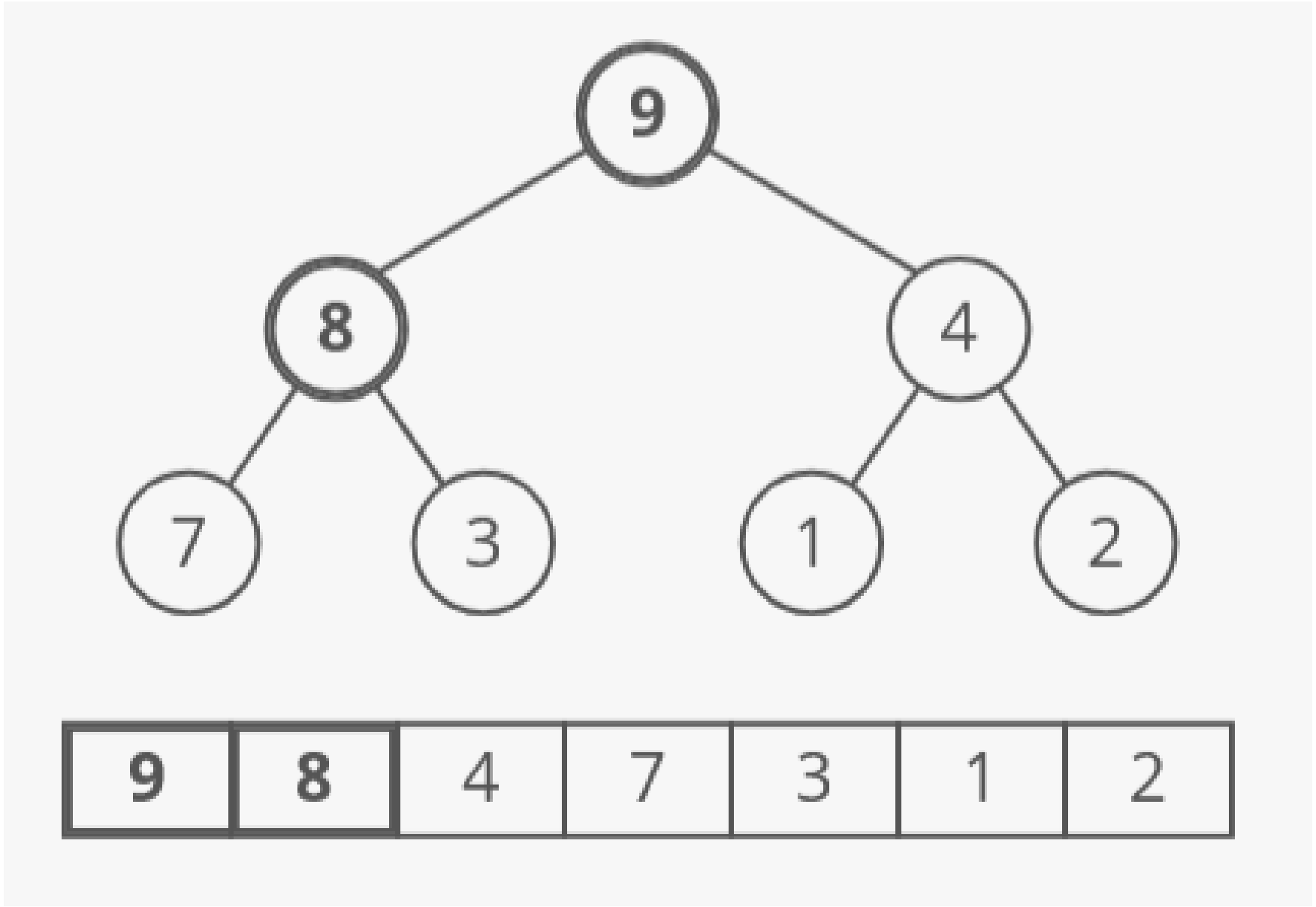

8
9
2
7
3
1
4
8
9
2
7
3
1
4


8
9
4
7
3
1
2
8
9
4
7
3
1
2


8
9
4
7
3
1
2
8
9
4
7
3
1
2

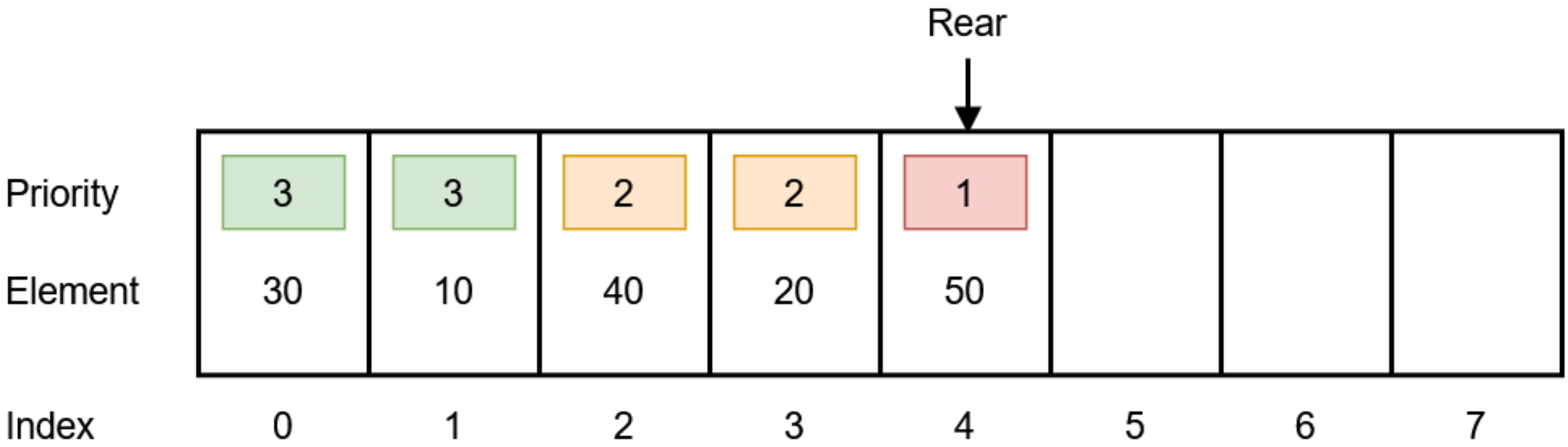

8
9
4
7
3
1
2
8
9
4
7
3
1
2

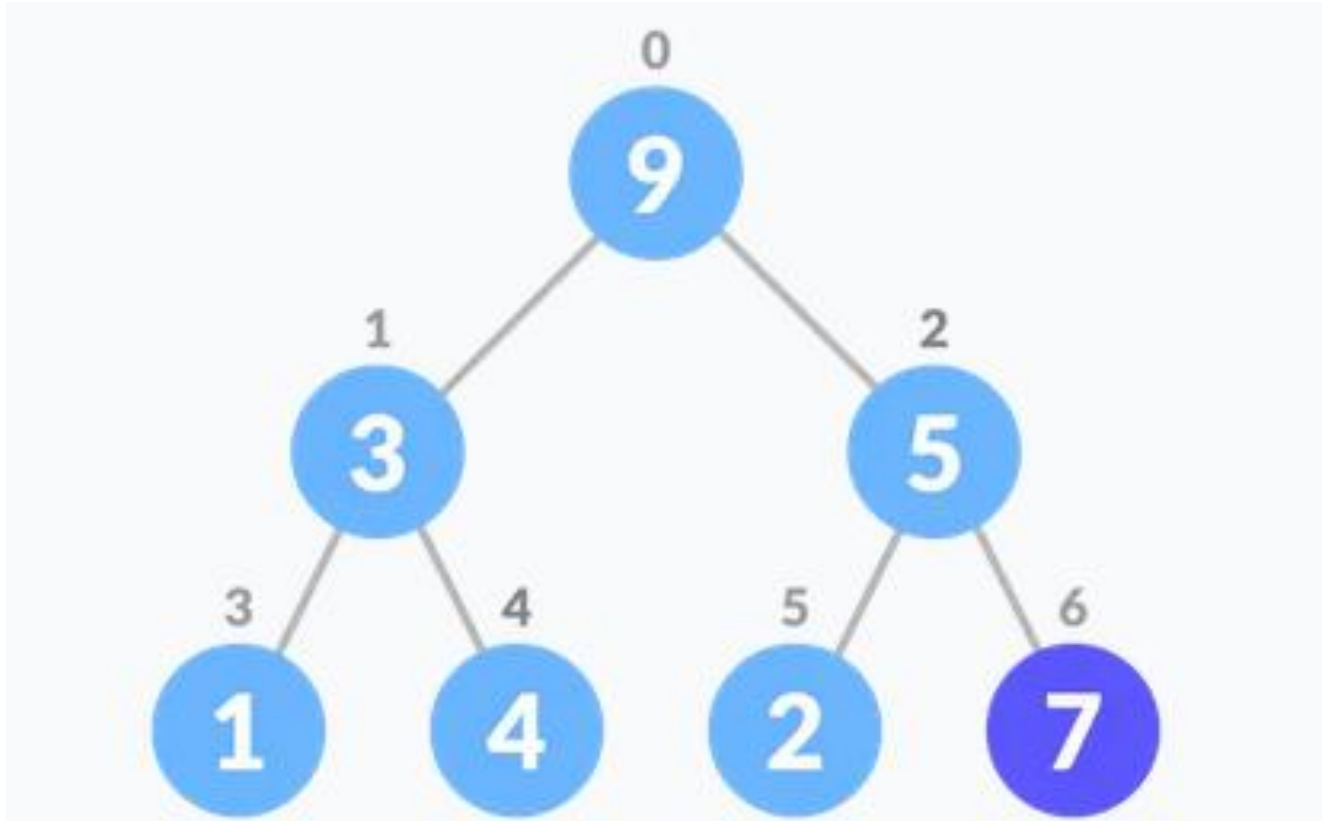

8
9
4
7
3
1
2
8
9
4
7
3
1
2


9
8
4
7
3
1
2
9
8
4
7
3
1
2

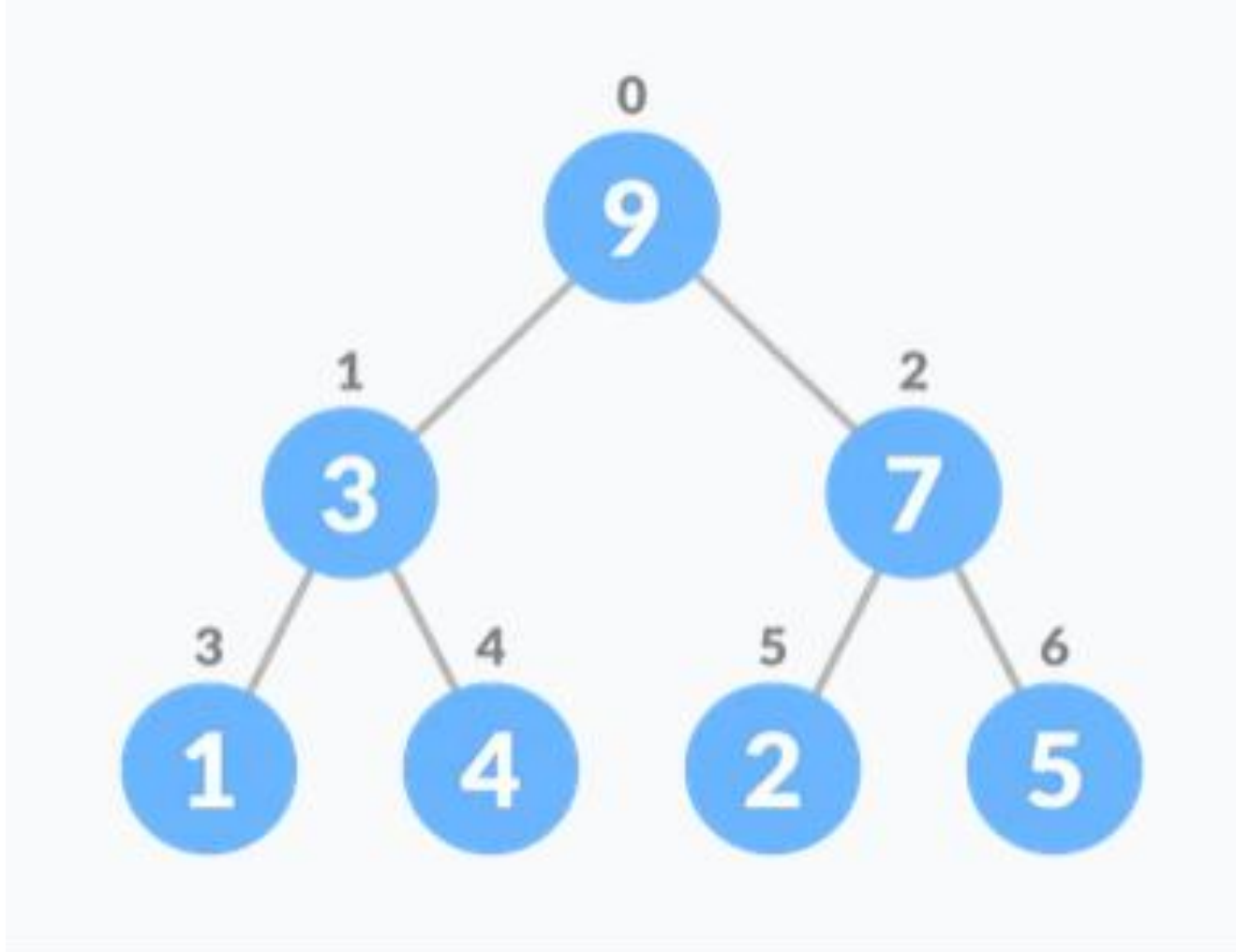


Priority queue


Rear
Priority
Element
Index
3
3
2
2
1
30
10
40
20
50
0
1
2
3
4
5
6
7



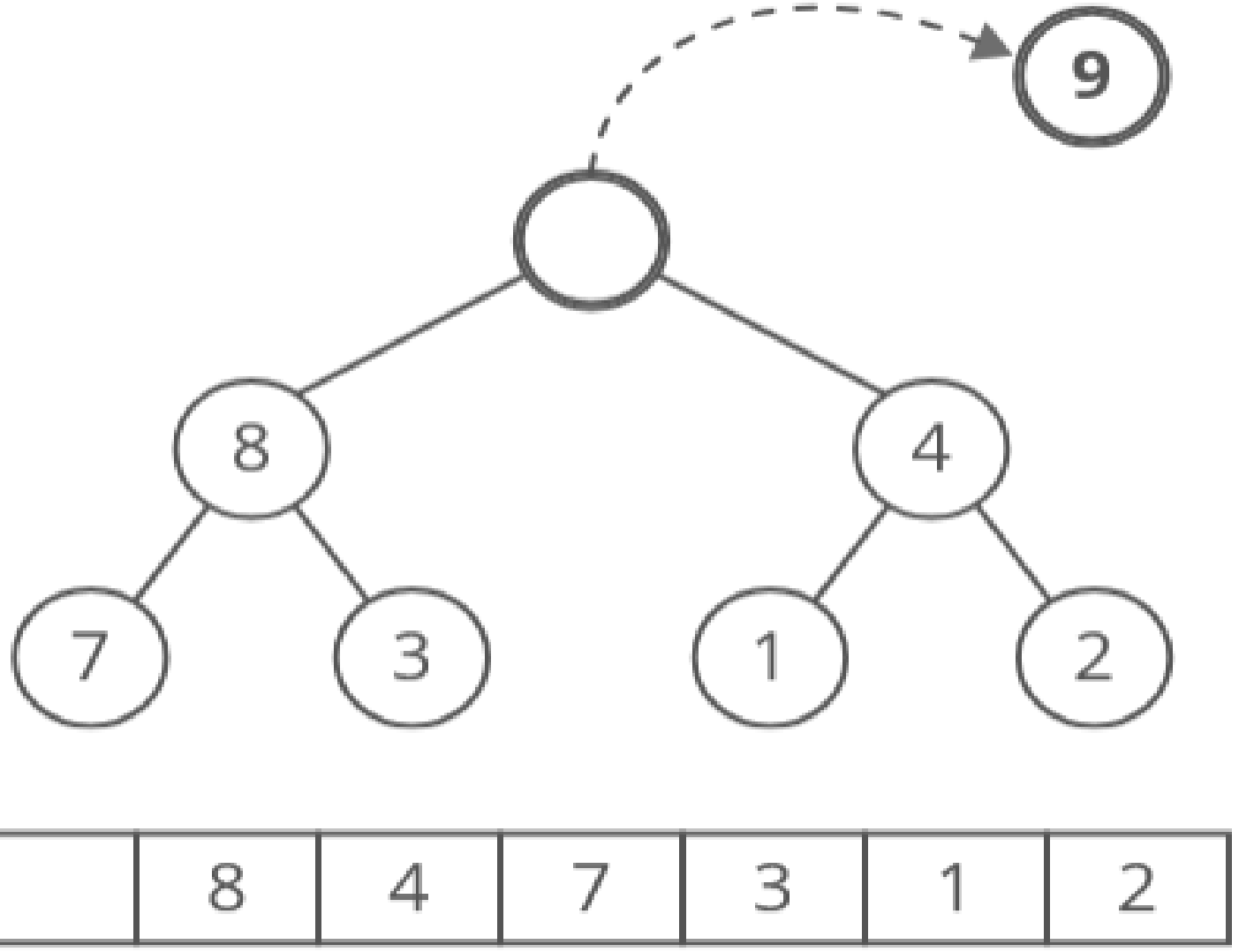
Insert an element at the end of the queue



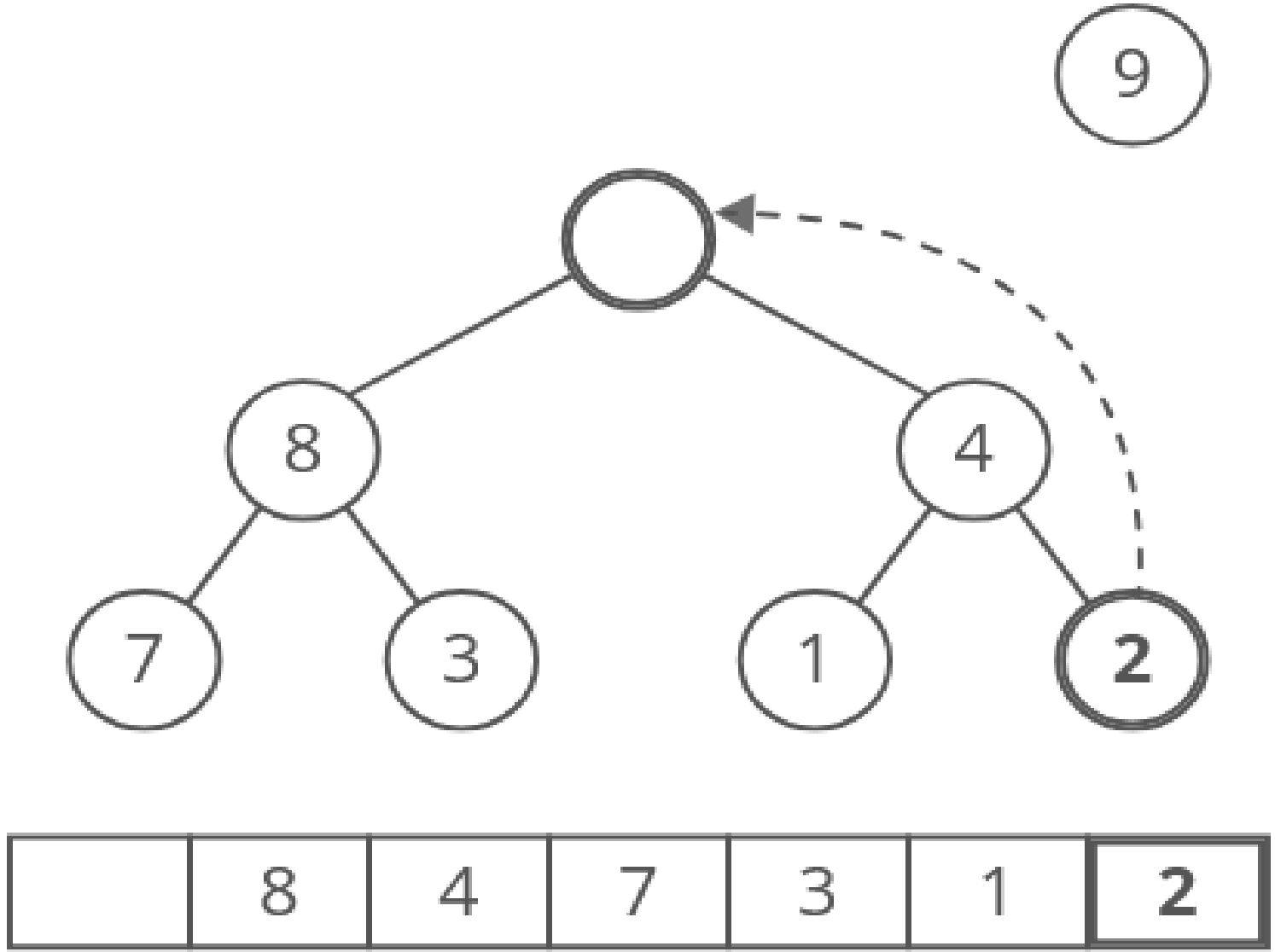
Heapify after insertion

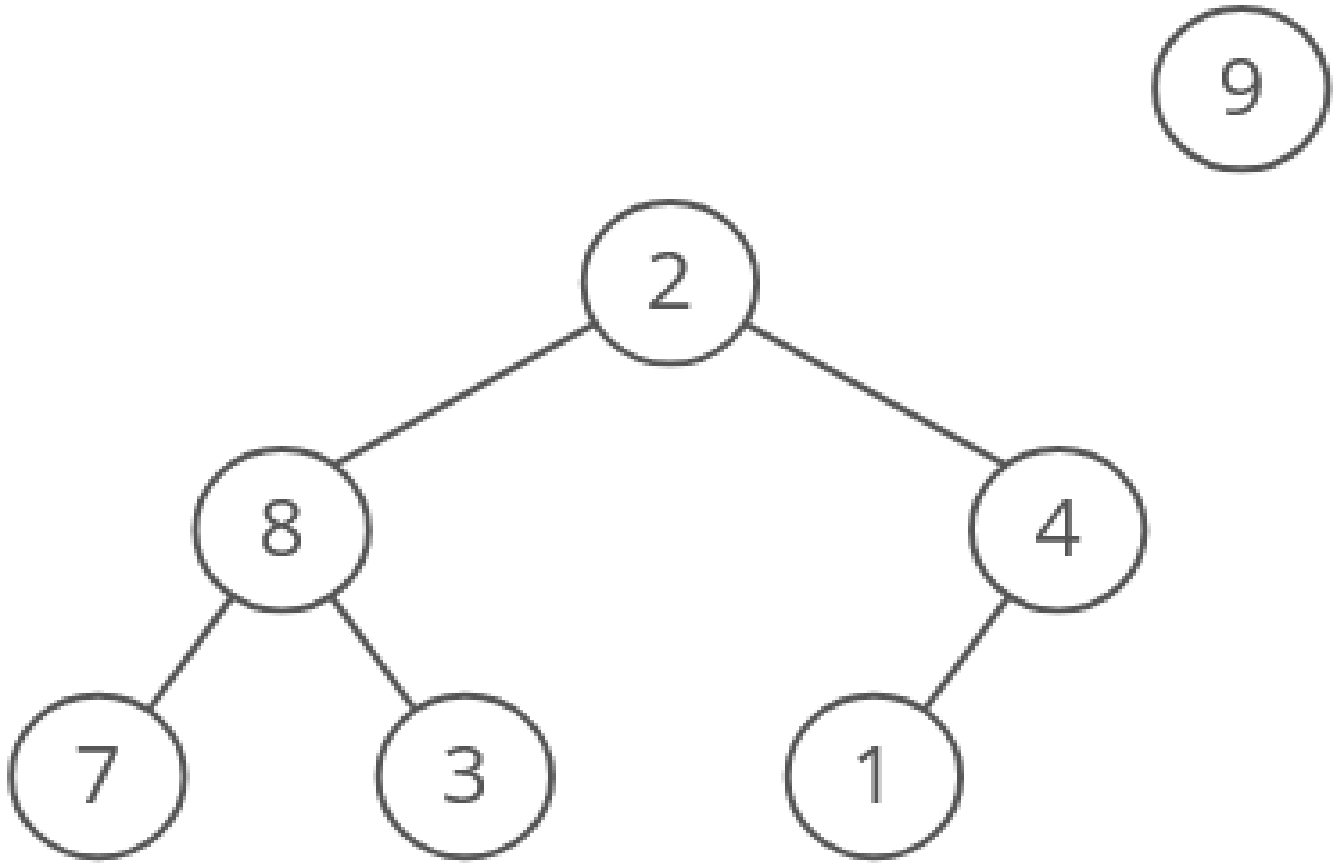
Removing the largest element



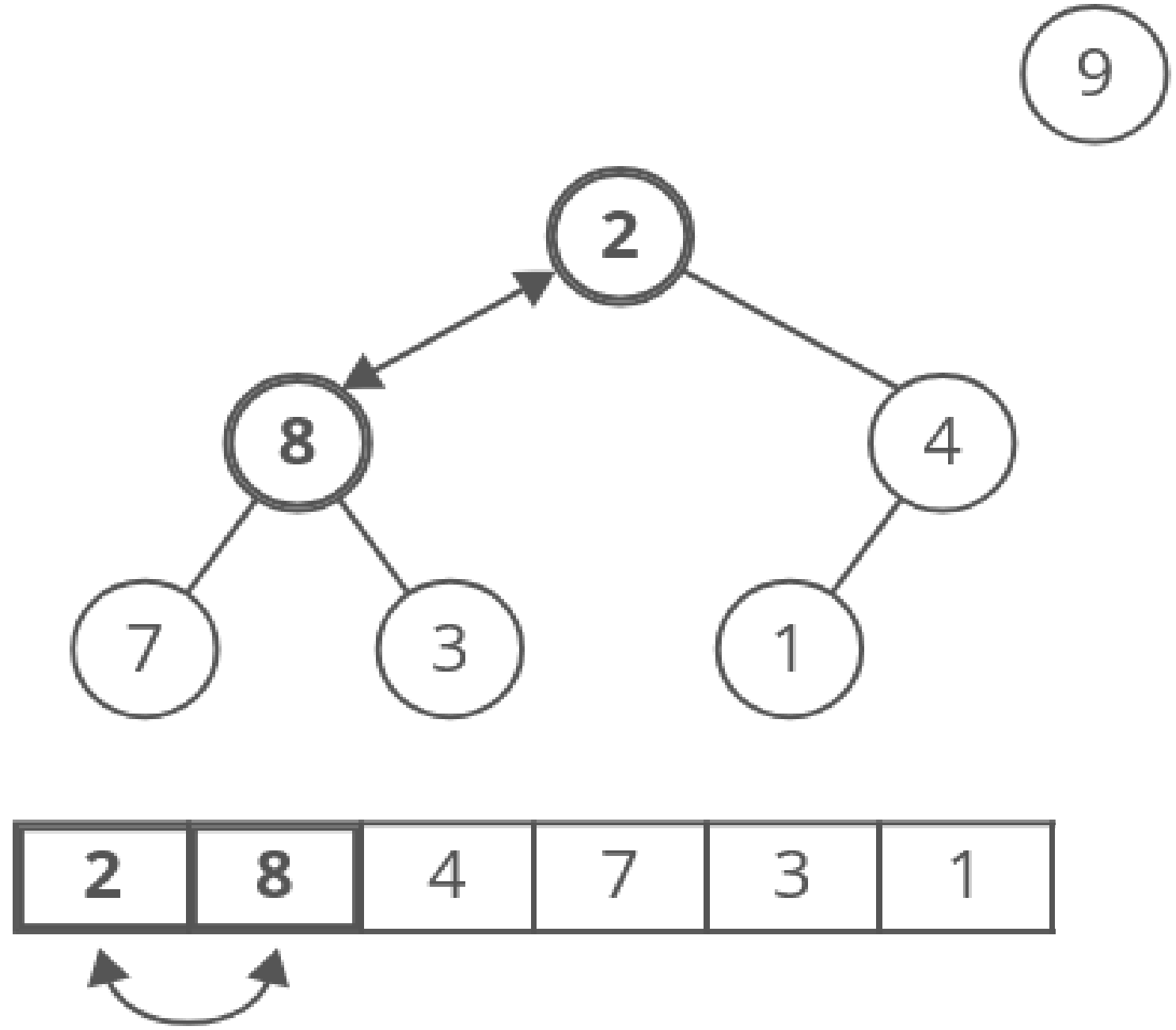


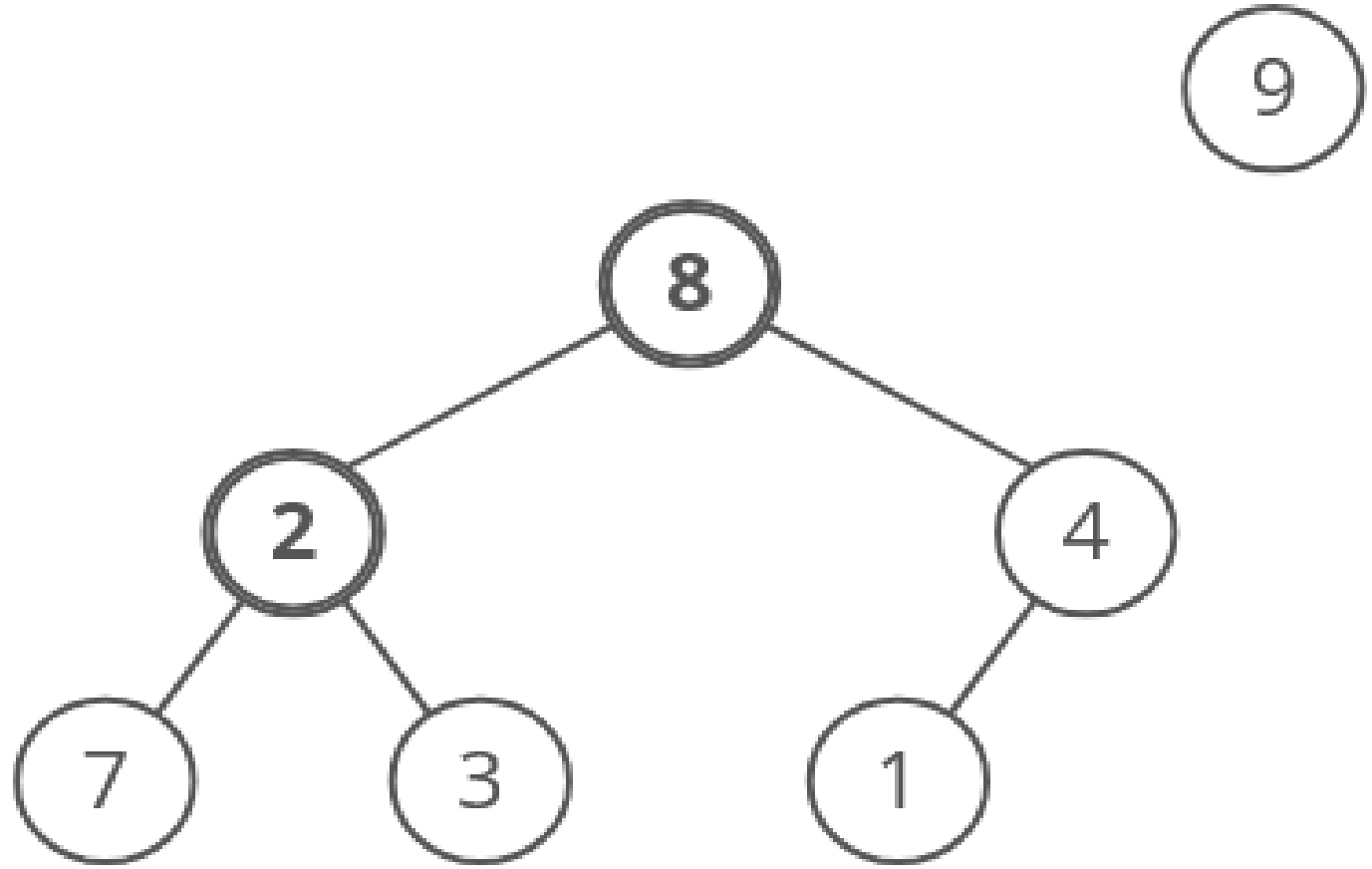
	8	4	7	3	1	2


9
8
4
7
3
1
2
8
4
7
3
1
2

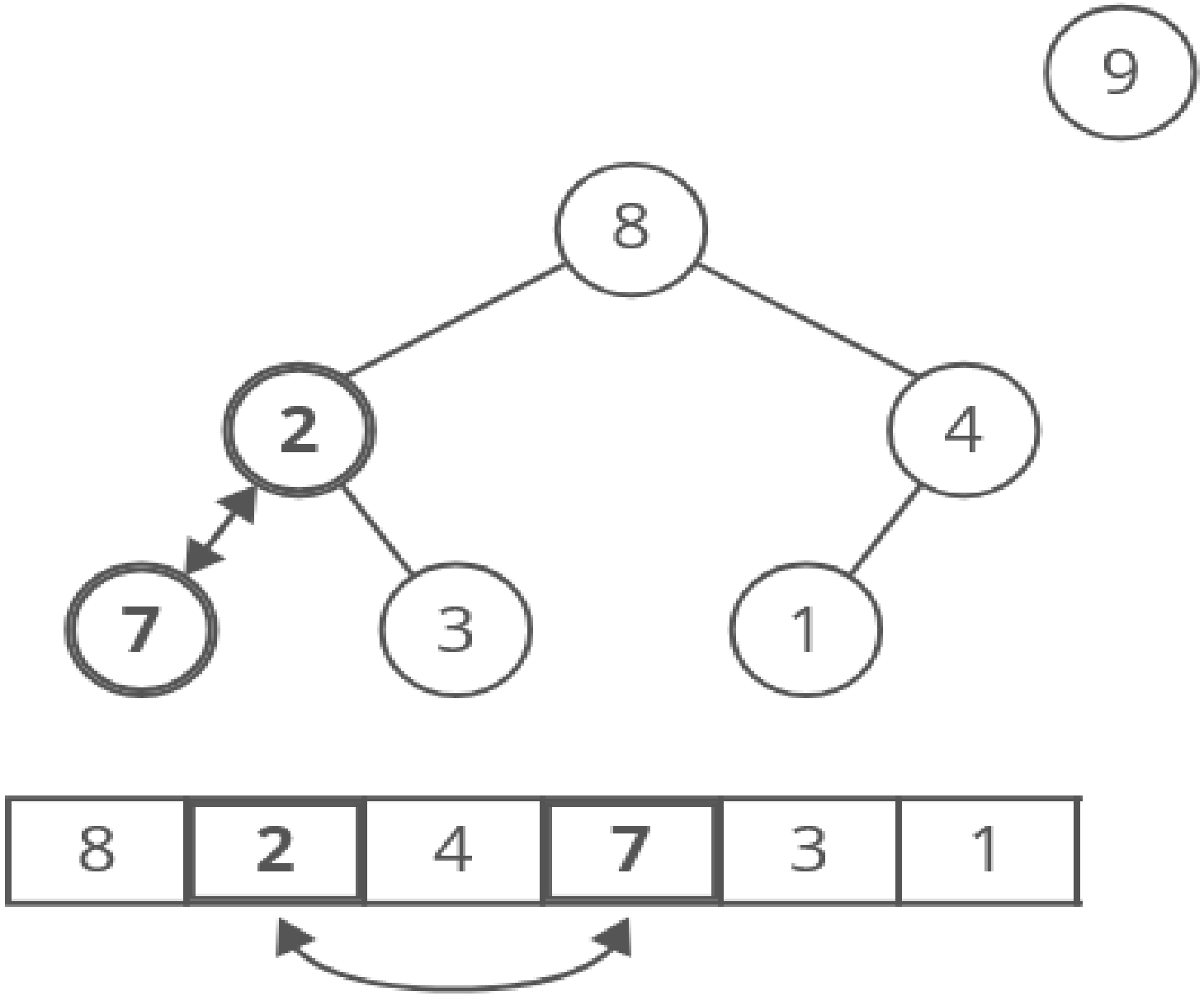


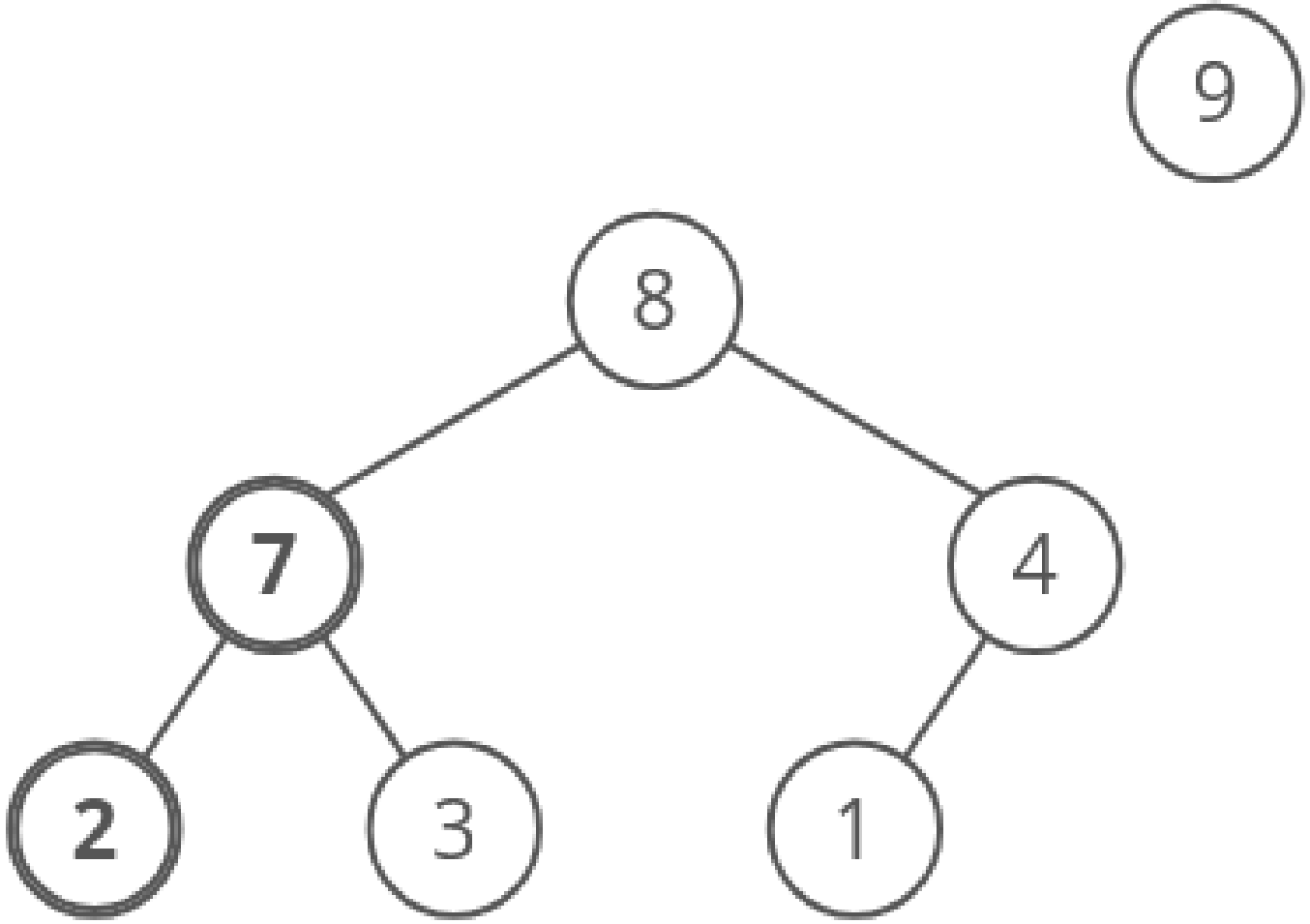
2	8	4	7	3	1


9
2
8
4
7
3
1
2
8
4
7
3
1



8	2	4	7	3	1


9
8
2
4
7
3
1
8
2
4
7
3
1



8	**7**	4	**2**	3	1

Time complexity of heap operations

- Inserting an element – *O(lg(n))*
- Removing an element – *O(lg(n))*

What about building the Initial heap?

- With the right implementation – *O(n)*
  - *Homework for extra points: find the proof!*